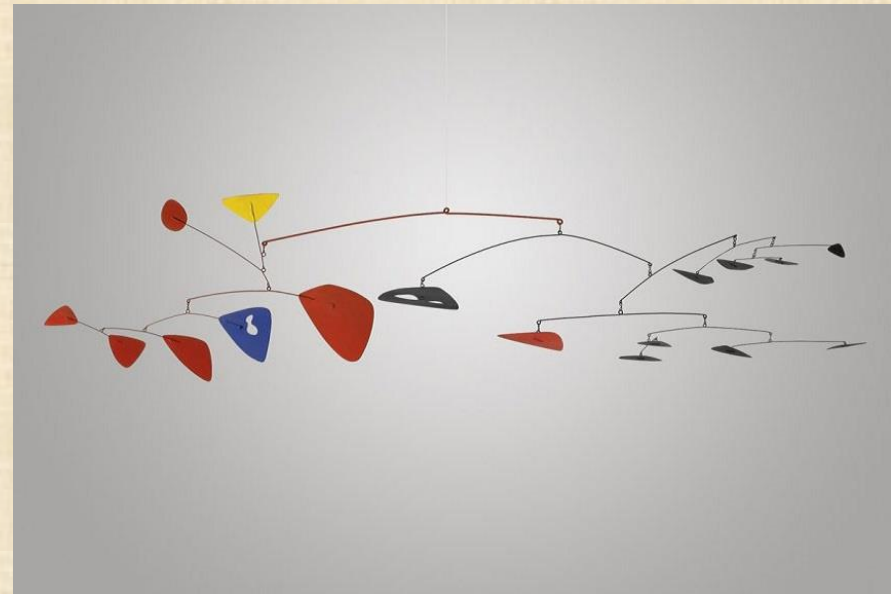


Class 8

Augmented Data

AVL Tree

Union Find



Last Lecture

- ▷ Binary Search Trees
 - ▷ Find, Insert, Delete, Min, Max, Successor, Predecessor, Join, Split operations
 - ▷ Tree traversals

Today

- ▷ Other operations
- ▷ Augmented data
- ▷ Balanced BST – AVL trees
- ▷ Disjoint Sets / Union-Find

Additional Operations

- ▷ Rank - What is the *rank* of a given key?
- ▷ Select - What is the key with a *rank* k ?
- ▷ Compute the *sum* of values of all elements that are larger than a given key?
- ▷ What is the average of the 10% lower keys?
- ▷

Selection

Return the element with *rank* k

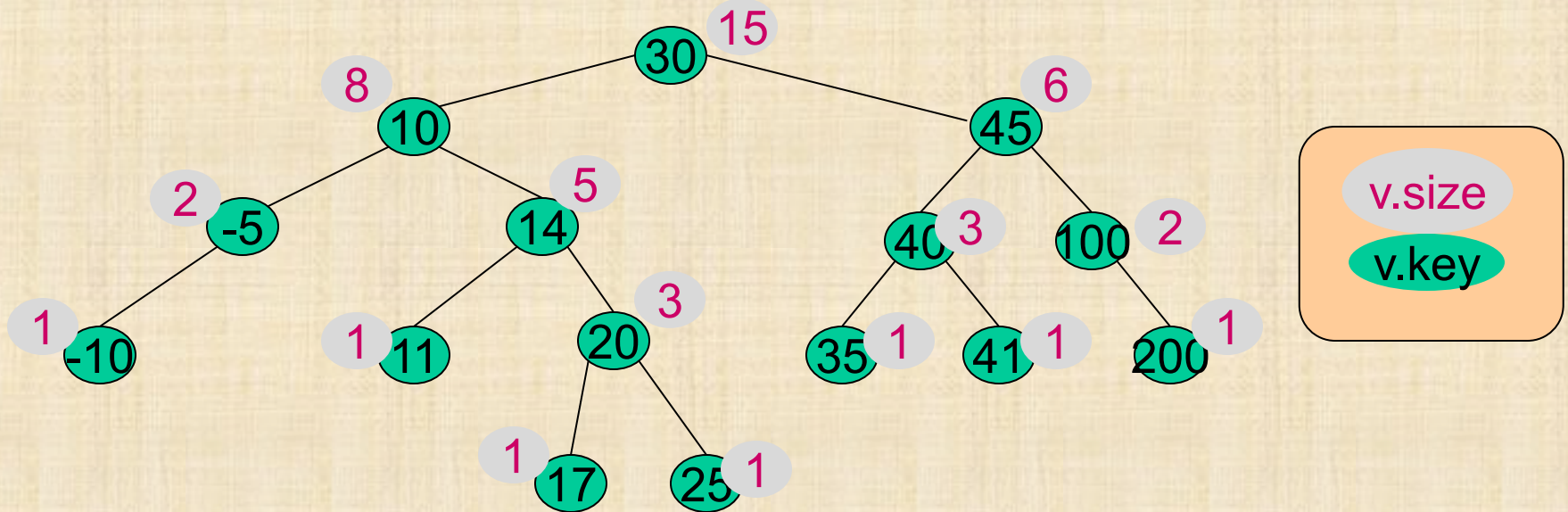
- ▷ For a sorted array A : $A[k]$
- ▷ For an unsorted array:
 - ▷ Quick selection
 - ▷ Median-of-medians (MMSelect)
- ▷ What about a BST?
 - ▷ Naïve solution: worst case $\Theta(n)$
 - ▷ We will see a more efficient solution using **augmented data**

Augmented Data

- ▷ Additional data in each node can be used to efficiently answer different questions
- ▷ Requires updating this data whenever the tree is updated (*insert, delete, ...*).
 - ▷ These updates should have the same complexity as insert / delete in a BST ($\Theta(h)$)

Size augmentation

- ▷ Add one more entry per node
- ▷ $v.size$: the number of nodes in the subtree rooted at v
- ▷ Each node v stores: the key, the data, and $v.size$



Updating Size Field

▷ Node insert:

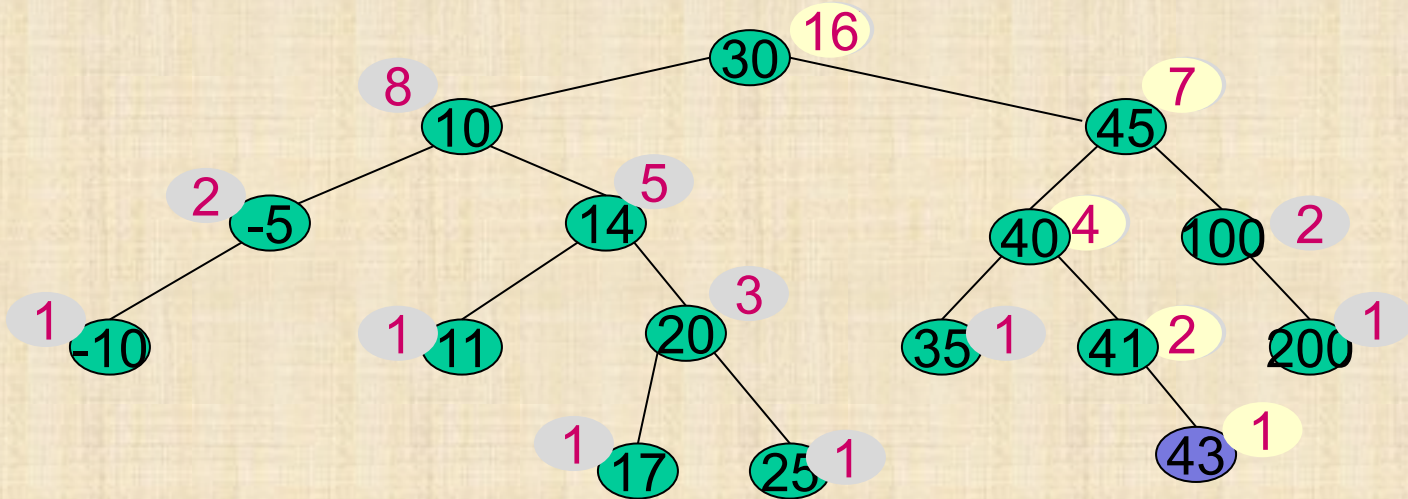
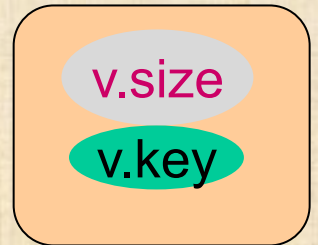
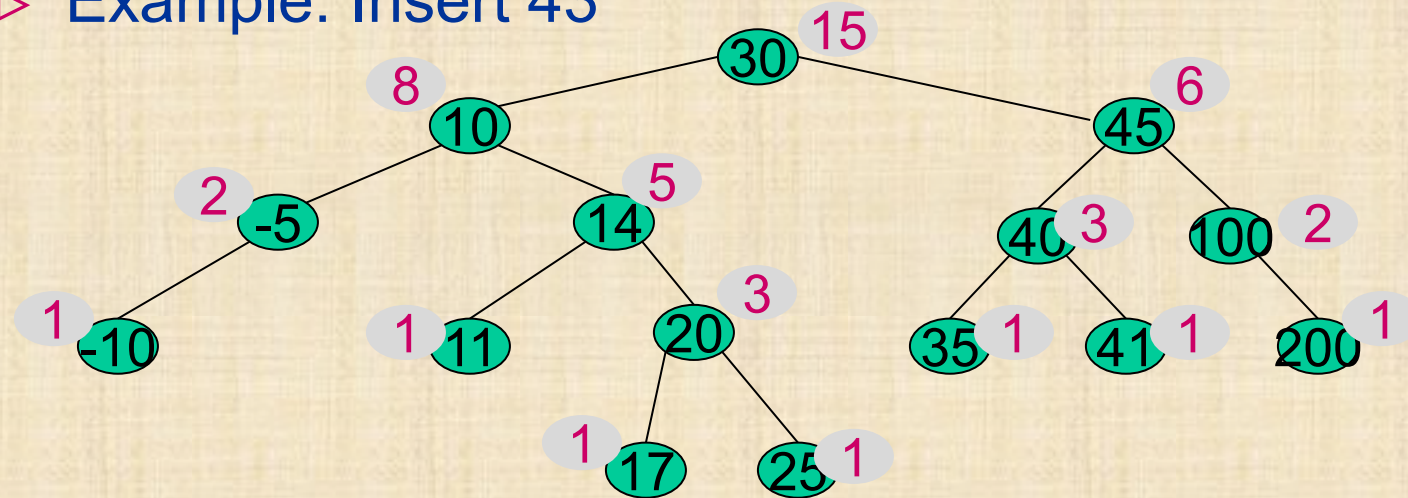
- ▷ A new node, x , is always a leaf: $x.size=1$
- ▷ Increase by 1 all $v.size$ along the path from x to the root

▷ Node delete:

- ▷ Decrease by 1 all $v.size$ along the path from the root to the node
- ▷ Note: in case of 2 children – the deleted node is the successor

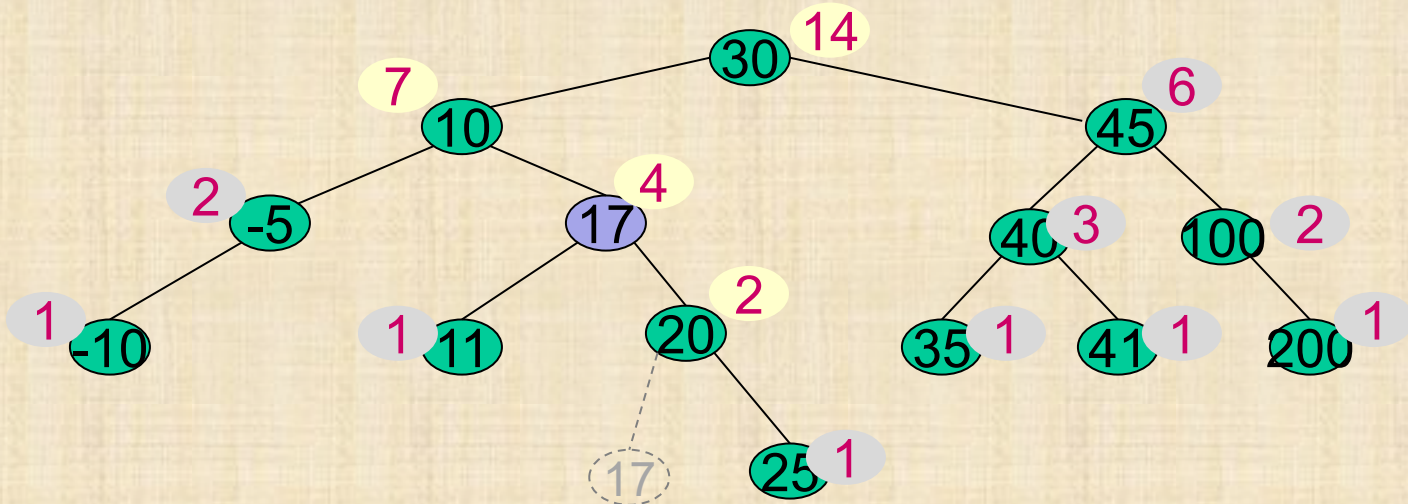
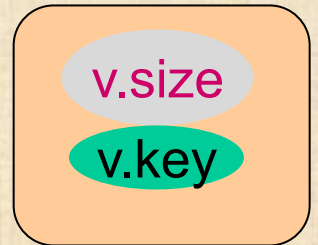
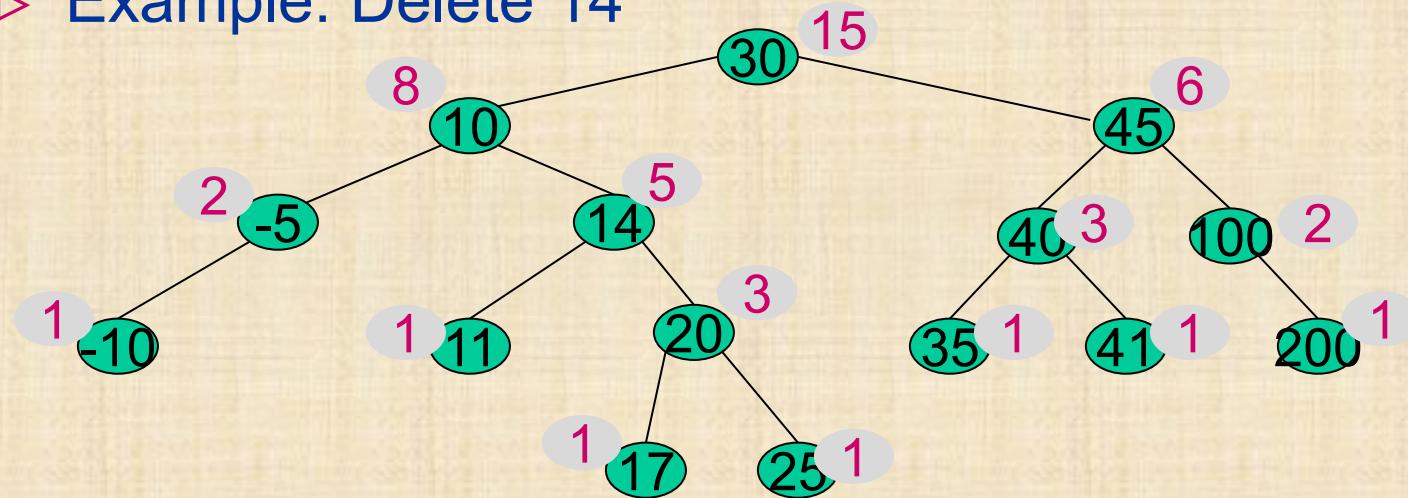
Updating Size Field

▷ Example: Insert 43



Updating Size Field

▷ Example: Delete 14



SELECT(T, k)

Goal: Return a pointer to the node containing the element with rank k in T

Idea:

▷ $r[x]$ - the rank of node x in the subtree rooted at x

▷ Use the size field:

$$r[root] = (root.left).size + 1$$

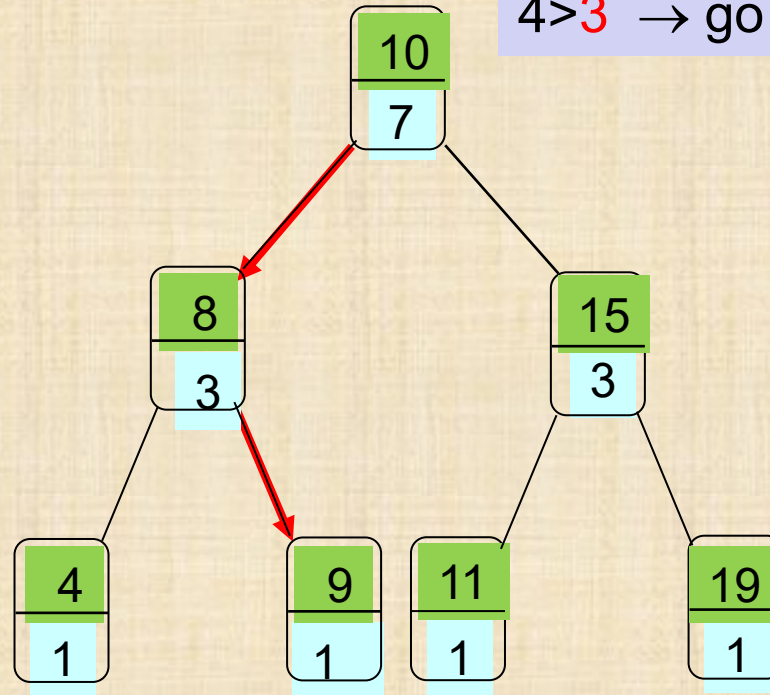
Example: $\text{SELECT}(T, k)$

Abused notations:
We use the 'key' as a
pointer to the node

$\text{SELECT}('10', 3)$

$\text{SELECT}('10', 3)$
 $r[10] = 3 + 1 = 4$
 $4 > 3 \rightarrow \text{go left}$

$\text{SELECT}('8', 3)$
 $r[8] = 1 + 1 = 2$
 $2 < 3 \rightarrow \text{go right}$



$\text{SELECT}('9', 1)$
 $r[9] = 0 + 1 = 1$.
Found! $\rightarrow 1 = 3 - 2$

SELECT(T, k)

Initial call: SELECT(T, k)

SELECT(x, k)

if ($x = \text{null}$), return null

if $x.\text{left} \neq \text{null}$

 then $r \leftarrow (x.\text{left}).\text{size} + 1$

else $r \leftarrow 1$

if $k = r$

 then return x

else if $k < r$

 then return *SELECT*($x.\text{left}, k$)

else return *SELECT*($x.\text{right}, k - r$)

x : a node

k : rank in a subtree
with x as a root

% rank of x in the
subtree rooted at x

Time complexity of Select: $O(h)$

RANK(T, x)

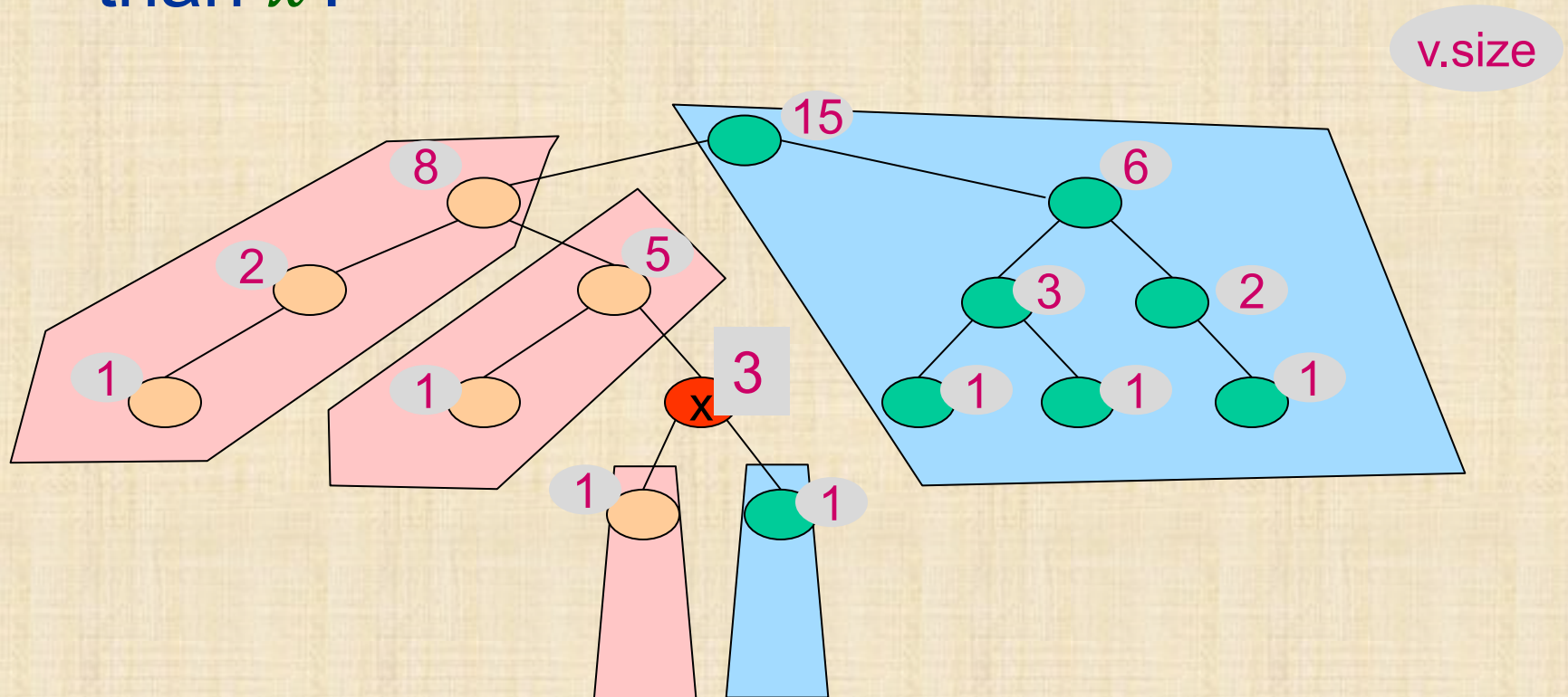
Goal: given a pointer x to a node in a BST T return the rank of x in T

Idea:

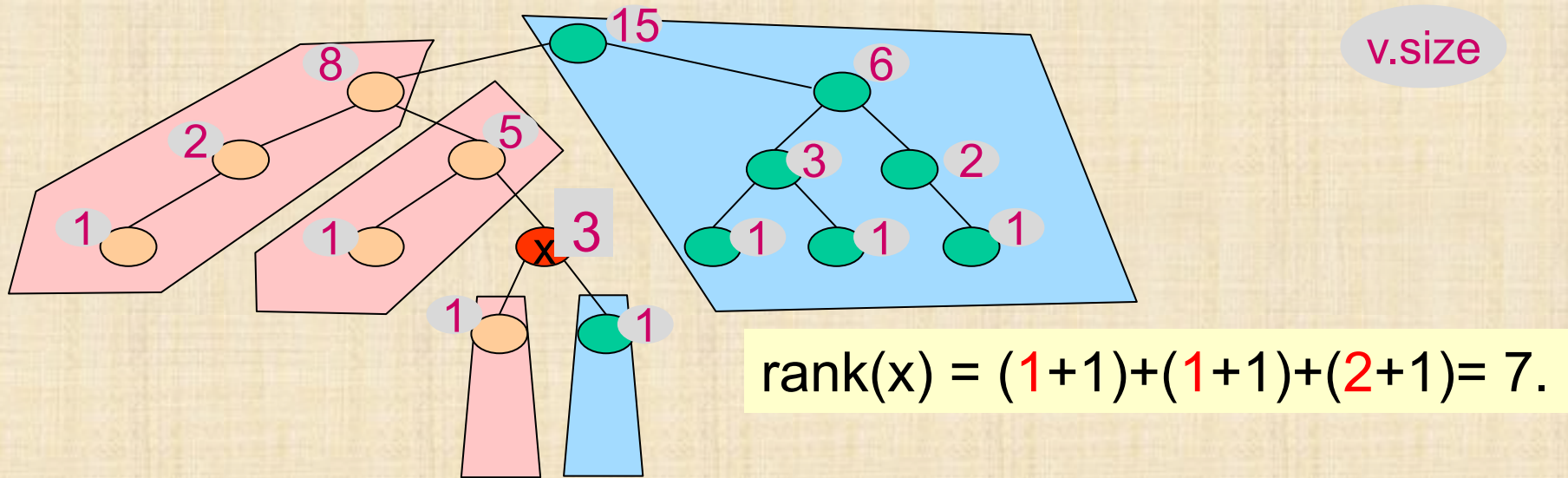
- ▷ Use size augmentation
- ▷ Count the number of keys in T that are smaller than the key of x :
 - ▷ The size of the left subtree of x
 - ▷ What else?

$\text{RANK}(T, x)$

- ▷ Where in the tree are the values smaller than x ?



Rank(T, x)



1. Add the elements in the left subtree of x , $+1$ for x itself
2. Consider the path p from x to the root:

For every v in p (including x): If v is a right child, then add the size of v 's parent's left subtree $+1$ for the parent

Rank(T, x)

T : root of the tree
 x : a node

RANK(T, x)

If $x.\text{left} \neq \text{null}$ % r : the rank of x in the tree rooted in x

 then $r \leftarrow (x.\text{left}).\text{size} + 1$

 else $r \leftarrow 1$

$y \leftarrow x$

while $y.\text{parent} \neq \text{null}$ do % y a pointer that traverse up the tree

 if $y = (y.\text{parent}).\text{right}$

 then if $(y.\text{parent}).\text{left} \neq \text{null}$

 then $r \leftarrow r + ((y.\text{parent}).\text{left}).\text{size} + 1$ % rank of y in the

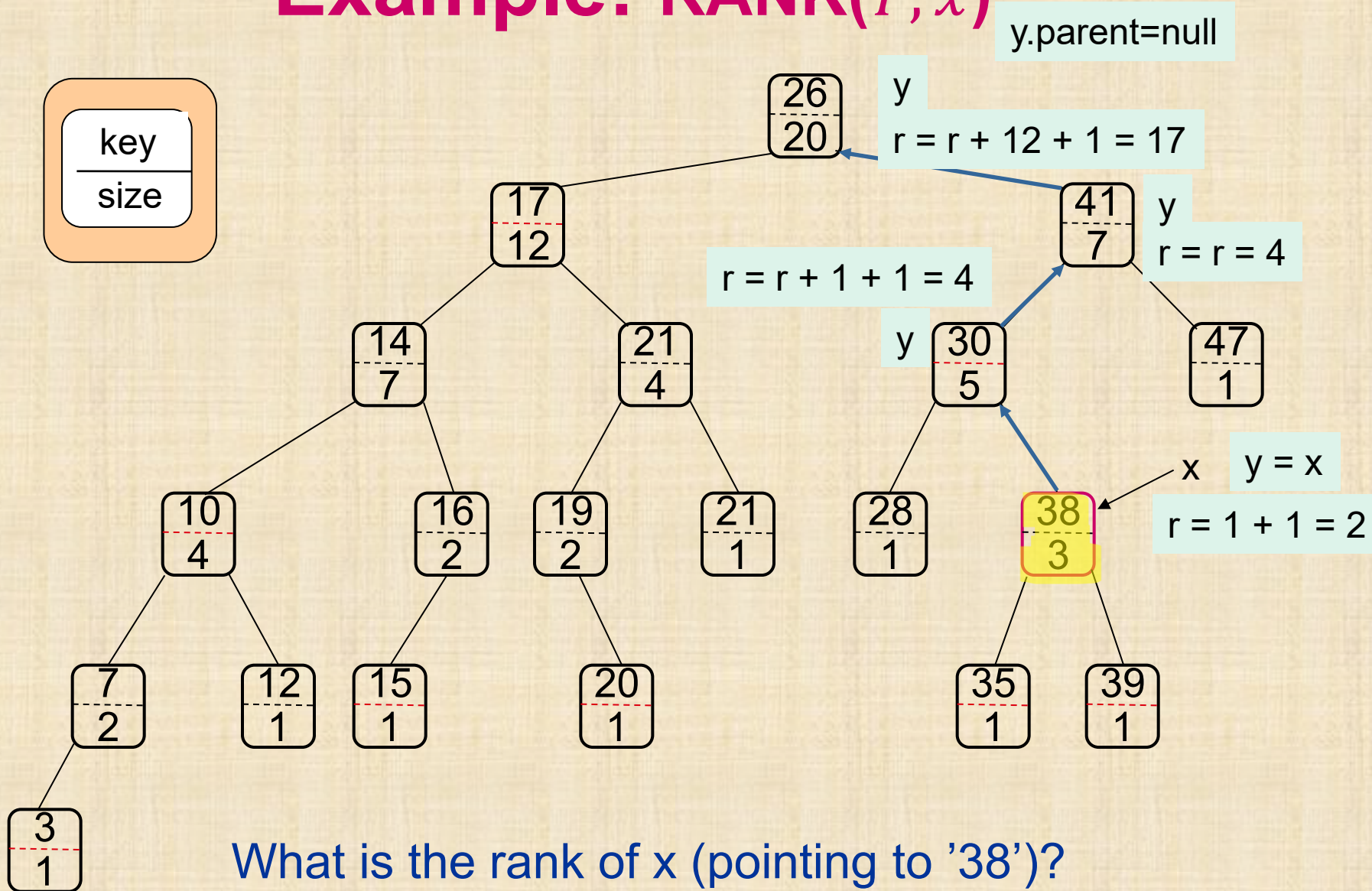
 else $r \leftarrow r + 1$ tree rooted at y

$y \leftarrow y.\text{parent}$

return r

Time complexity?

Example: $\text{RANK}(T, x)$



Sum of Keys $< x$

Goal: given a value x return the sum of keys smaller than x in the tree

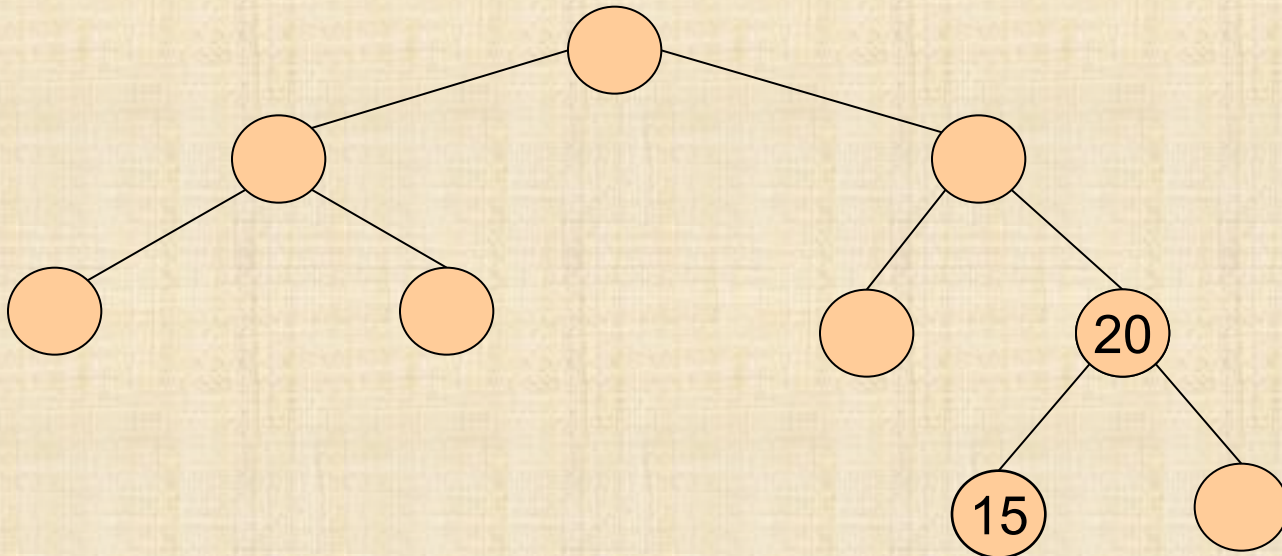
Idea:

- ▶ Use augmented BST data structure:
 - ▶ Store in v a field $v.sum$ = the sum of all keys in the subtree rooted at v
- ▶ Use it to answer sum-related queries

Sum of Keys $< x$

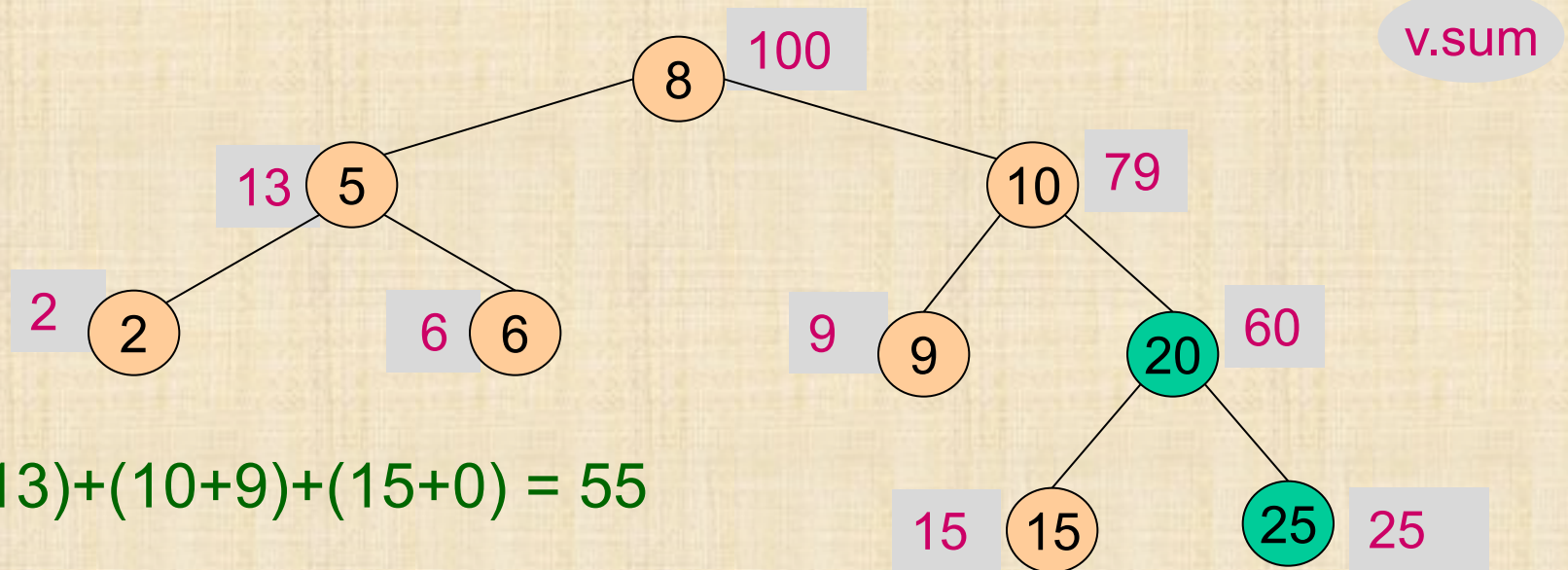
▷ Where are the keys that are < 17 ?

Where is 17 ?



Sum of Keys < x

- ▷ Where are the **keys** that are < 17?
- ▷ What is the sum of keys that are < 17?



SumSmaller

SumSmaller (x,k)

if $x == \text{null}$, then return 0

if $k == x.\text{key}$

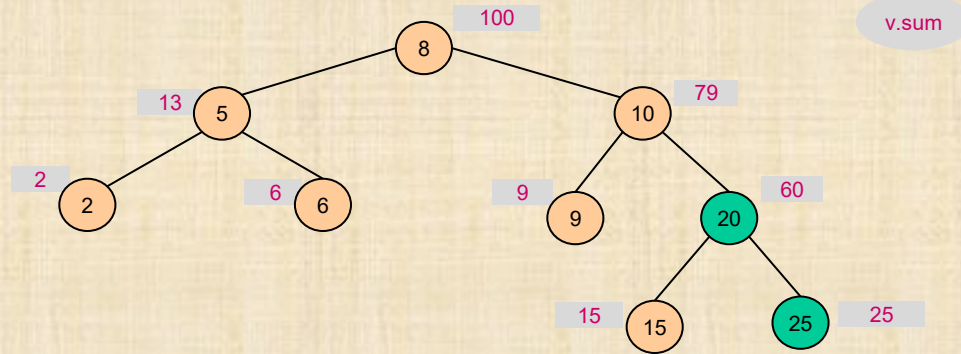
then $s \leftarrow (x.\text{left}).\text{sum}$

elseif $k < x.\text{key}$

then $s \leftarrow \text{SumSmaller}(x.\text{left}, k)$

else $s \leftarrow x.\text{key} + (x.\text{left}).\text{sum} + \text{SumSmaller}(x.\text{right}, k)$

return s



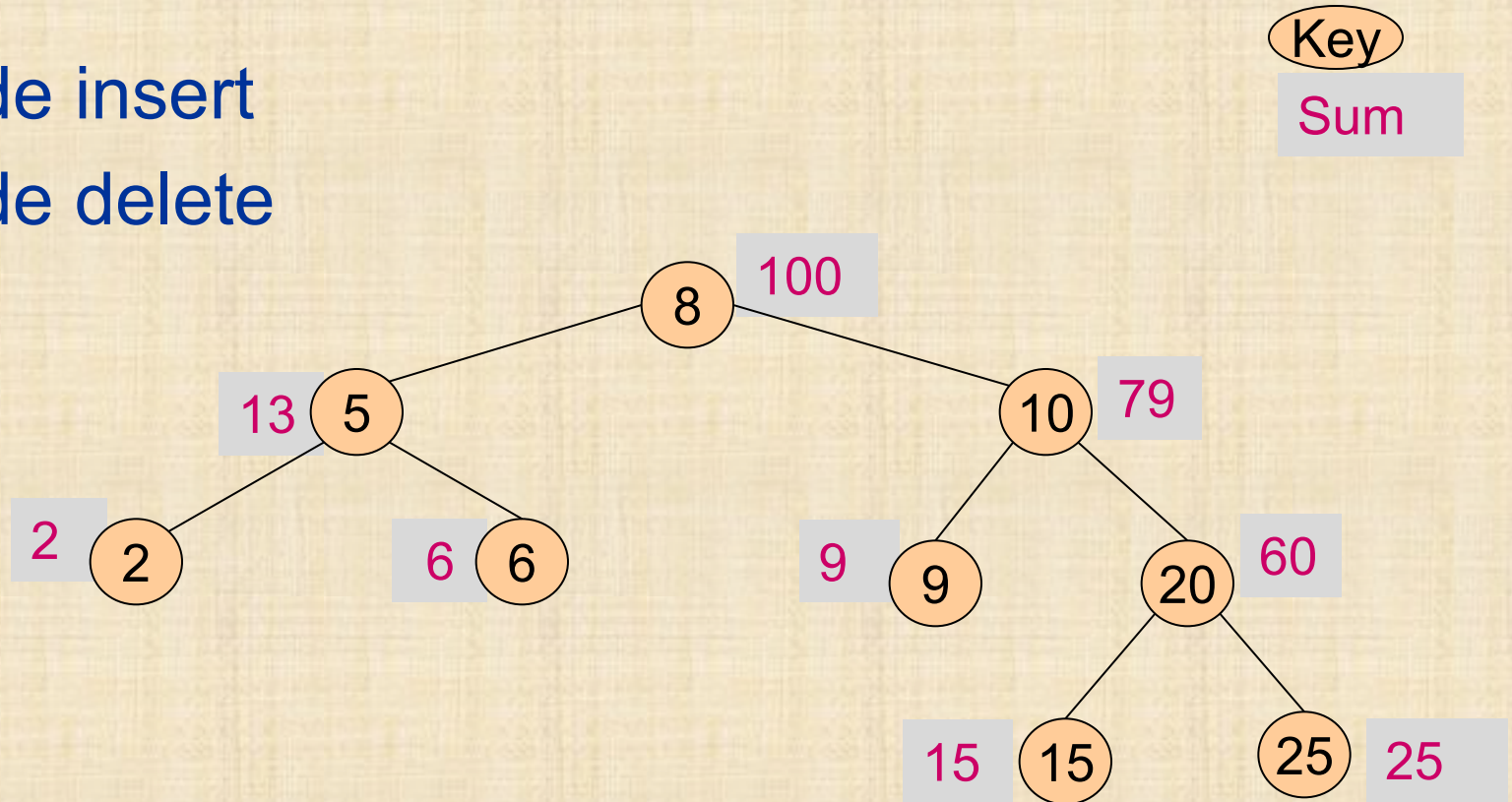
Initial call: *SumSmaller*(T,k)

Worst case running time: $\Theta(h)$

T: root of the tree
x: a node
k: a key

Updating Sum Field

- ▷ Node insert
- ▷ Node delete



Augmenting BSTs Theorem

Theorem: Let f be a field that augments a BST.

If the value of f for each node x can be computed using only the information in x , $x.\text{left}$, $x.\text{right}$, $x.\text{left}.f$ and $x.\text{right}.f$, then it is possible to maintain the values of f in all nodes during insertions and deletions in $O(h)$ time.

Proof Sketch: just like in size/sum augmentation.

Time Complexity

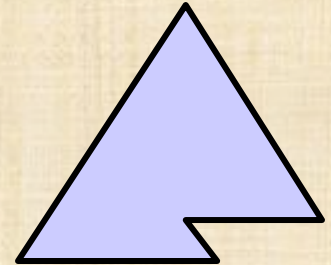
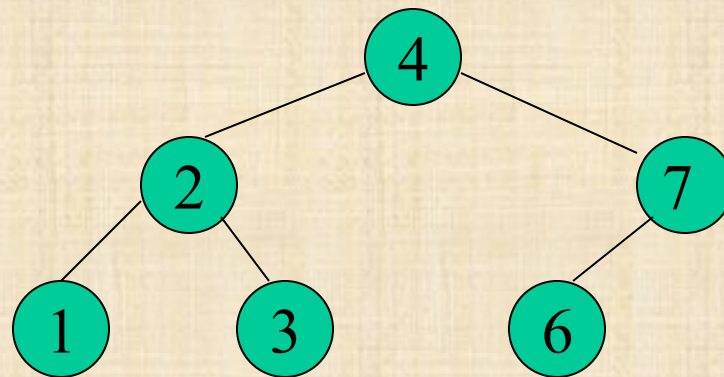
- ▷ All operations we considered so far have $O(h)$ time complexity where h is height of the tree
- ▷ Worst case: h might be $\Omega(n)$
 - ▷ When?
- ▷ Best case: h is $O(\log n)$
 - ▷ When?

Balanced Tree

- ▷ Height is $h = \Theta(\log n)$
- ▷ All the operations we saw take:
 $\Theta(h) = \Theta(\log n)$ time

Balanced Tree: (bad) Solution

- ▷ Always keep the elements in a full or almost complete binary tree



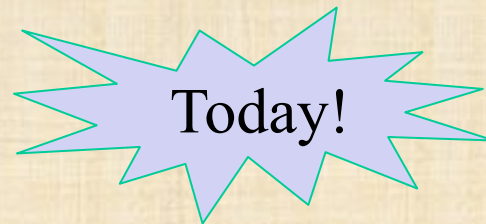
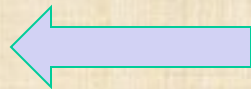
Ensures that the height is $\leq \log n$ but maintaining this property is not efficient.

Solution

- ▷ Keep the height of the tree to be bounded by $O(\log n)$ (instead of exactly $\log n$)
- ▷ This flexibility in the tree structure allows us to keep the time complexity in $O(h) = O(\log n)$

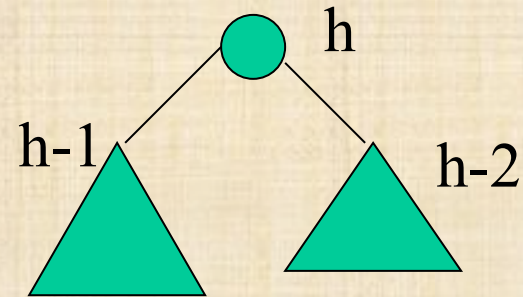
Several implementations:

- ▷ AVL trees
- ▷ B-trees (e.g., 2-3)
- ▷ Splay trees
- ▷ Red-black trees.



AVL-Trees

- ▷ Denote by $h(x)$ the height of the node x
- ▷ **AVL property:** The tree is considered balanced if for every node $x \in T$,
$$h(x.left) - h(x.right) \in \{-1, 0, 1\}$$
- ▷ Store $h(x)$ for each node x



We Need to Show

- ▷ Implementation
- ▷ Height of an AVL tree $O(\log n)$
- ▷ Insert and delete in $O(\log n)$

Implementation

- ▷ Use structures:
 - ▷ *height* is an augmented data field

key		
height		
Data		
L	P	R

- ▷ Must update when inserting or deleting a node in the tree
 - ▷ height of node only depends on height of its children
 - ▷ Only along the path of inserting/deleting

Height of an AVL Tree

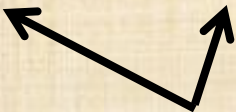
- ▷ Let $n_{min}(h)$ denote the **minimum** number of nodes in an AVL tree of height h
- ▷ We will prove (soon) that $n_{min}(h) \geq 2^{\frac{h}{2}}$
- ▷ *Since for every AVL tree of height h it holds that $n \geq n_{min}(h)$, we get that $n \geq 2^{\frac{h}{2}}$*
- ▷ Therefore, $2 \log n \geq h$

Claim: $n_{min}(h) \geq 2^{\frac{h}{2}}$

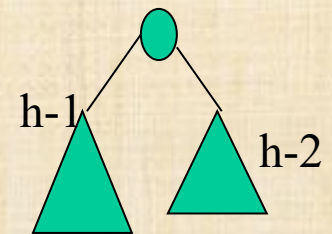
Proof: By induction on h

- ▷ Base case: $n_{min}(0) = 1, n_{min}(1) = 2 \geq 2^{\frac{1}{2}}$
- ▷ Induction hypothesis: $n_{min}(k) \geq 2^{\frac{k}{2}}, \forall k < h$
- ▷ Induction step: we prove for h

$$n_{min}(h) = n_{min}(h-1) + n_{min}(h-2) + 1$$

$$\geq 2^{\frac{h-1}{2}} + 2^{\frac{h-2}{2}} \geq 2 \cdot 2^{\frac{h-2}{2}} = 2^{\frac{h}{2}} \quad \blacksquare$$


induction hypothesis



A Tighter Bound

- ▷ Recall that

$$n_{min}(h) = n_{min}(h - 1) + n_{min}(h - 2) + 1$$

- ▷ Using properties of Fibonacci series, it is possible to prove a tighter bound:

$$n_{min}(h) \geq \varphi^h \quad (\varphi = \frac{1+\sqrt{5}}{2} \approx 1.62)$$

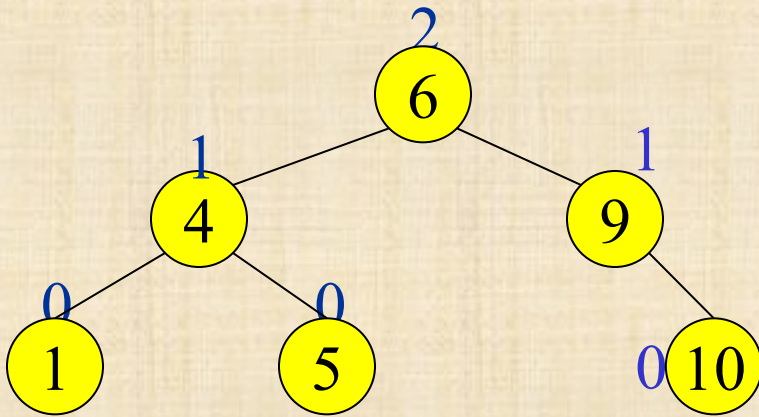
$$\text{That is, } h \leq \log_{\varphi} n = 1.44 \log_2 n$$

Maintaining the AVL property

- ▷ Modify Insert and Delete such that:
- ▷ For every node $x \in T$,
 $|h(x.left) - h(x.right)| \leq 1$
- ▷ The time for Insert and Delete remains
 $O(h) = O(\log n)$

How to maintain balance?

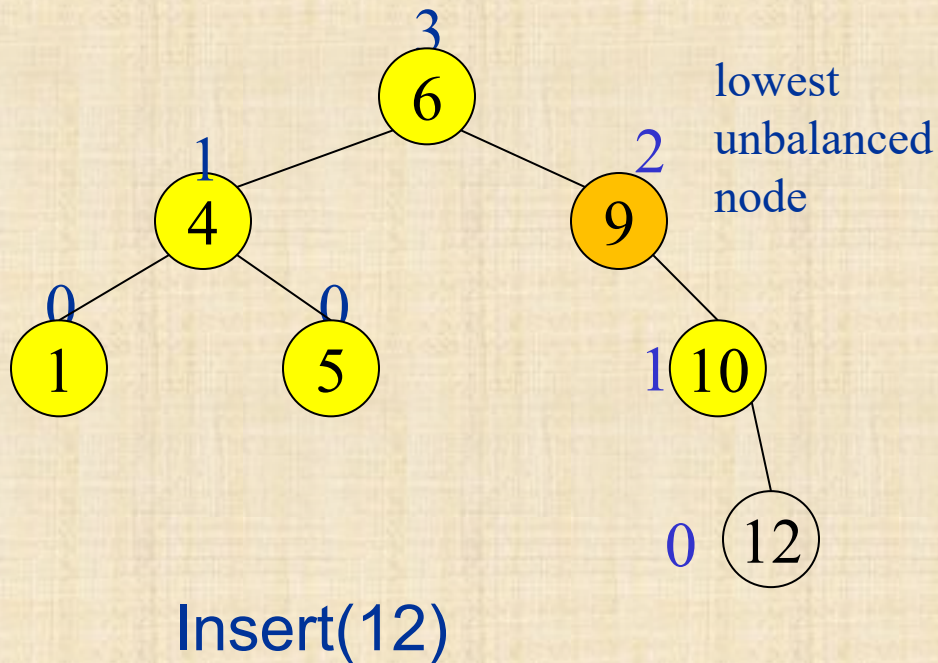
Example: insertion



Insert(12)

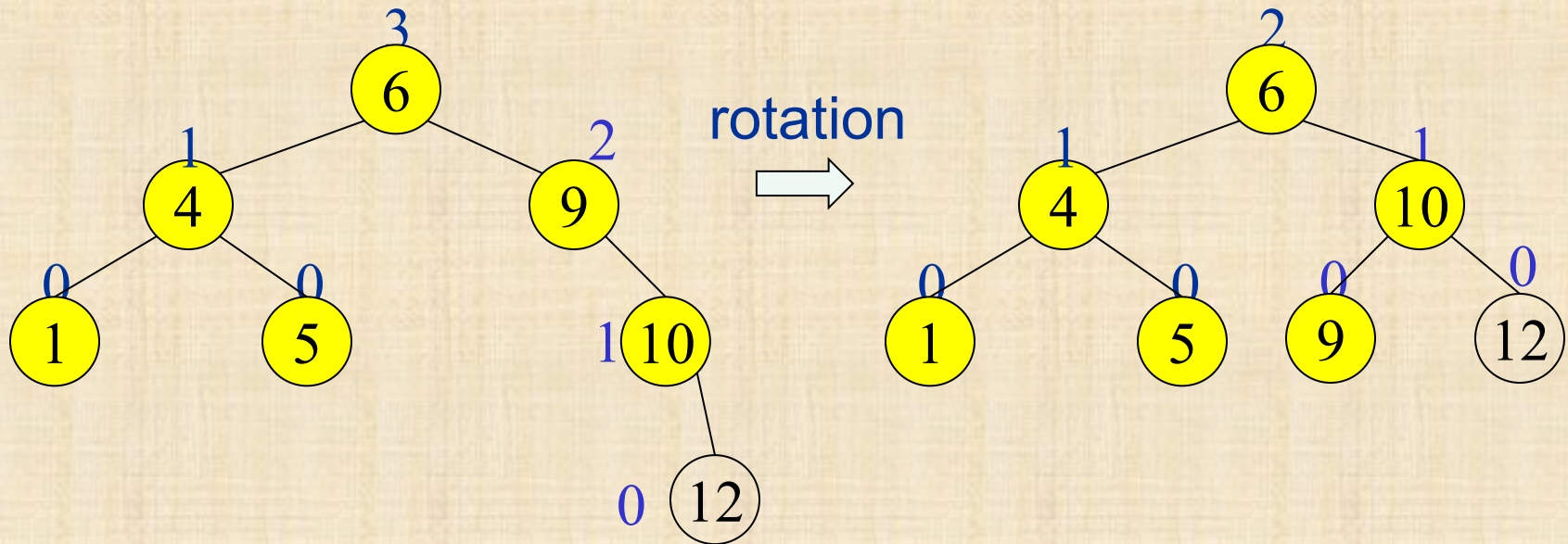
How to maintain balance?

Example: insertion



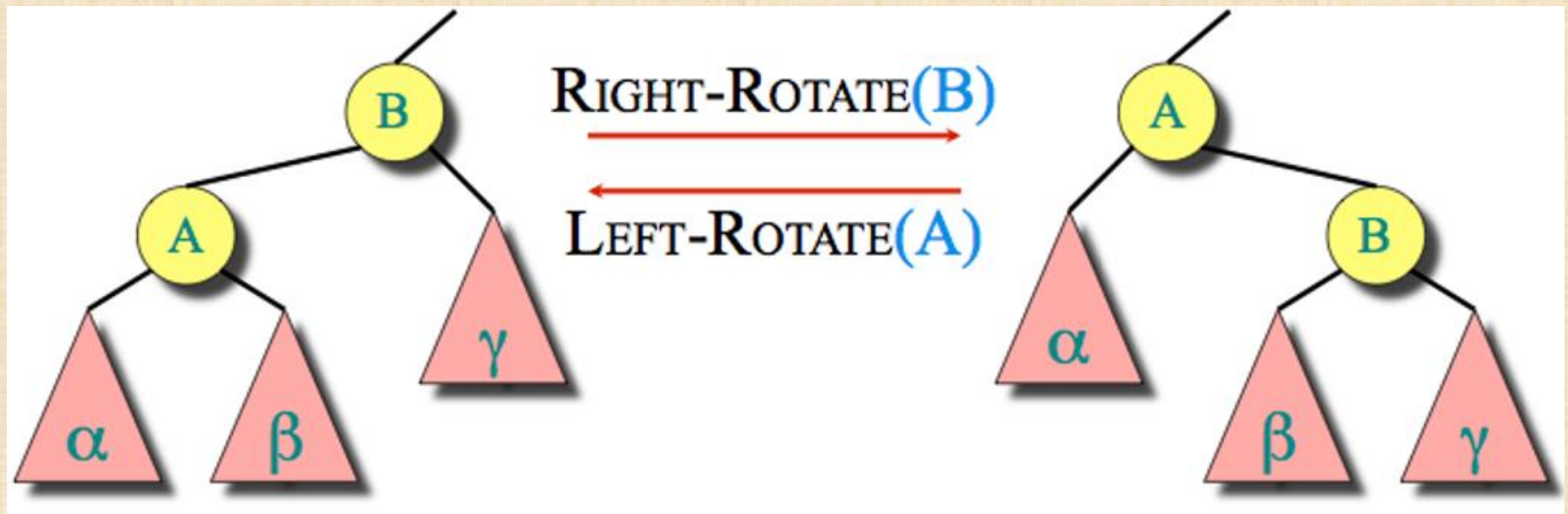
How to maintain balance?

Example: insertion



How do we keep the tree balanced?

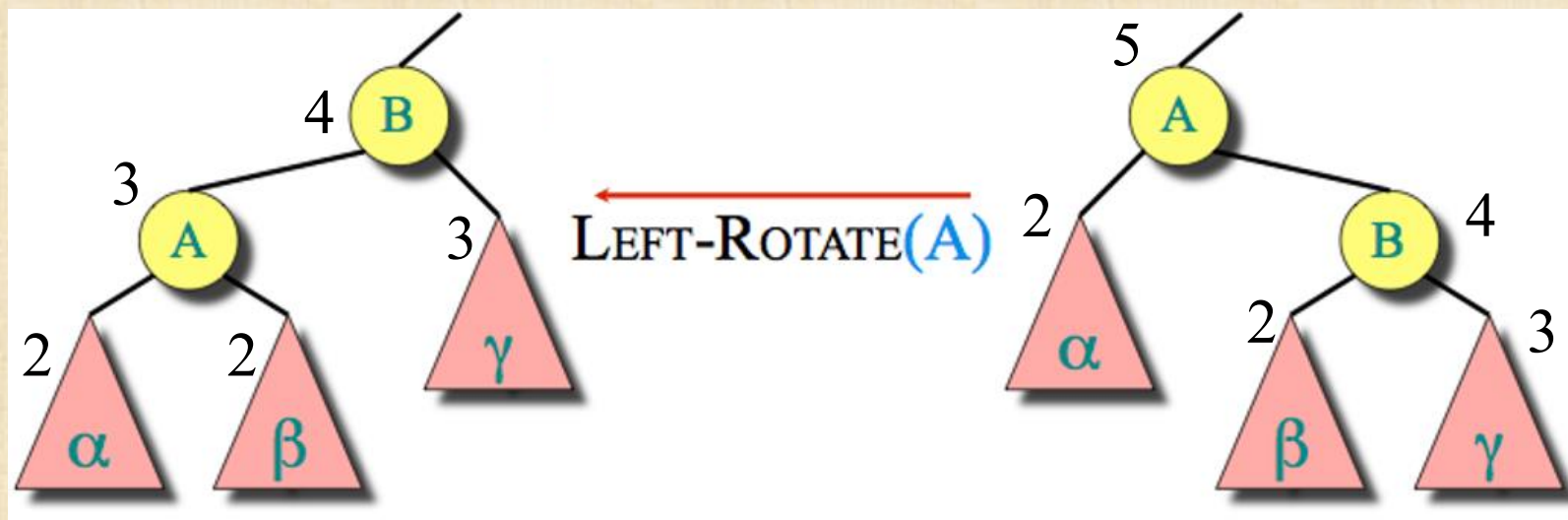
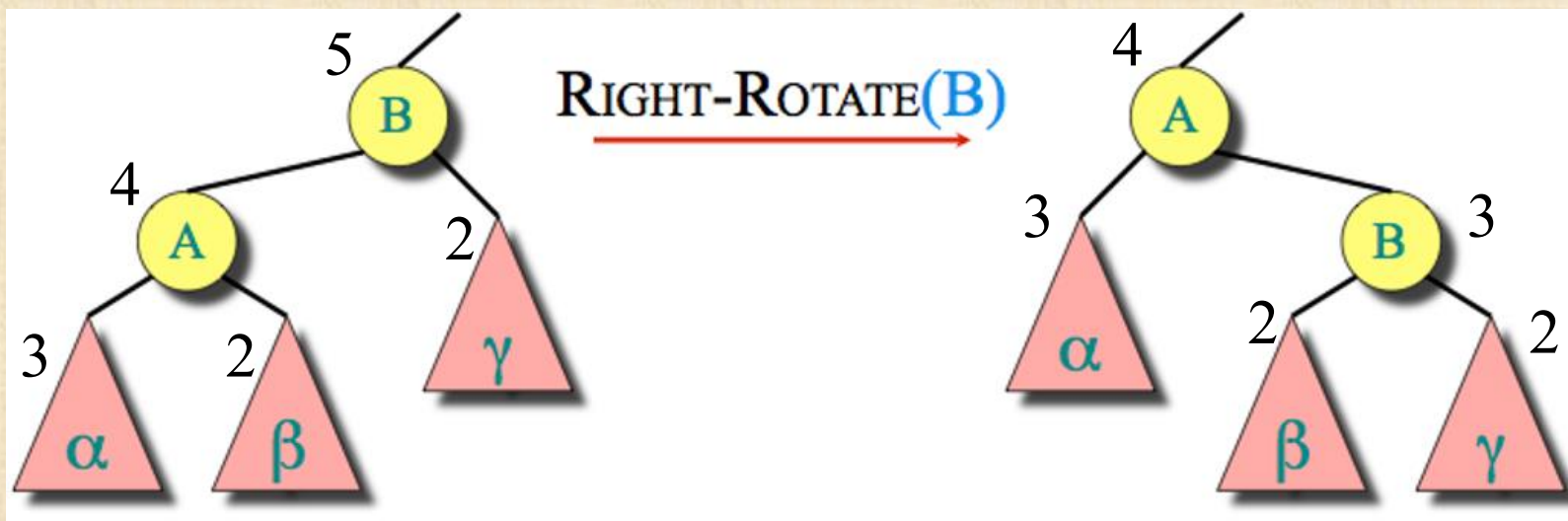
▷ Basic operations: Rotations



▷ Rotations maintain the BST property and the inorder ordering of keys:

$$\bullet a \in \alpha, b \in \beta, c \in \gamma \Rightarrow a \leq A \leq b \leq B \leq c.$$

How does a rotation affect the heights?

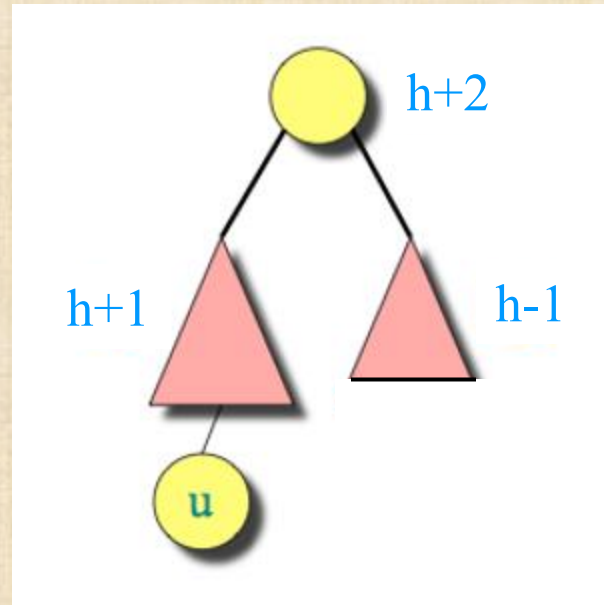
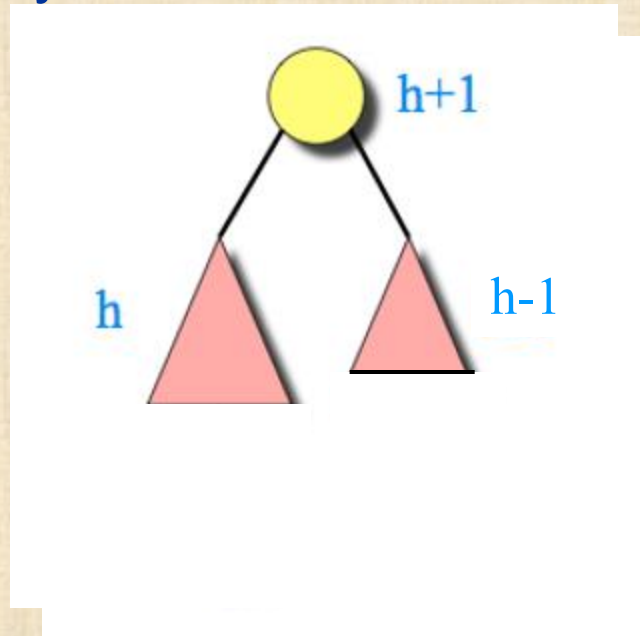


Balancing

- ▷ Whenever an operation that affects the height of some node is performed (insert, delete,...), we will restore the balance – by walking up the tree, from the affected node to the root, and making the required rotations.

Insertions

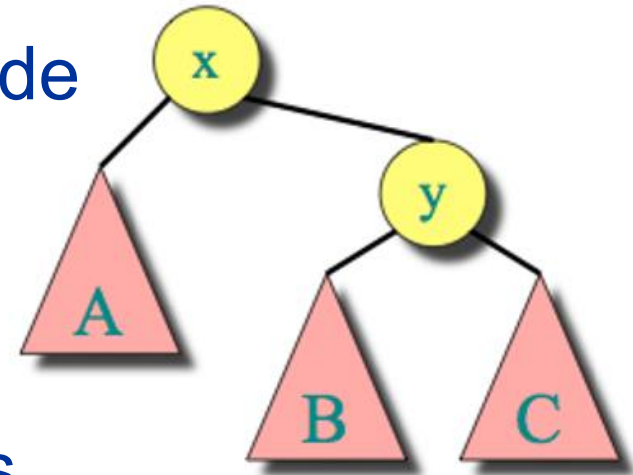
- Insert new node u as in a simple BST
- Go up the tree from the new node to the root, update the height fields and restore the balance by rotations, if needed



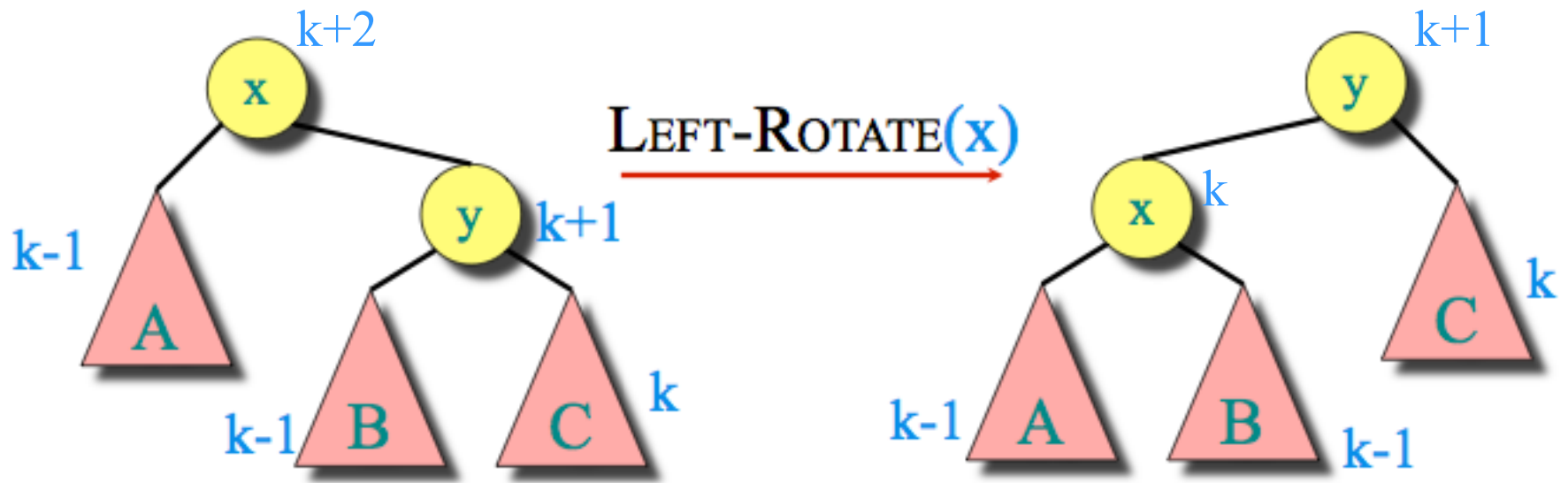
Example: $\text{insert}(u)$ causes imbalance in the root

Balancing

- ▷ Let x be the lowest “violating” node
- ▷ We will fix the subtree of x and move up
- ▷ Assume the right child of x is higher than the left child of x (x is called “right heavy”)
- ▷ Let y be the right child of x . There are 2 cases to consider:
 - Case 1: y is right-heavy or balanced ($h_B \leq h_C$)
 - Case 2: y is left-heavy ($h_B > h_C$)

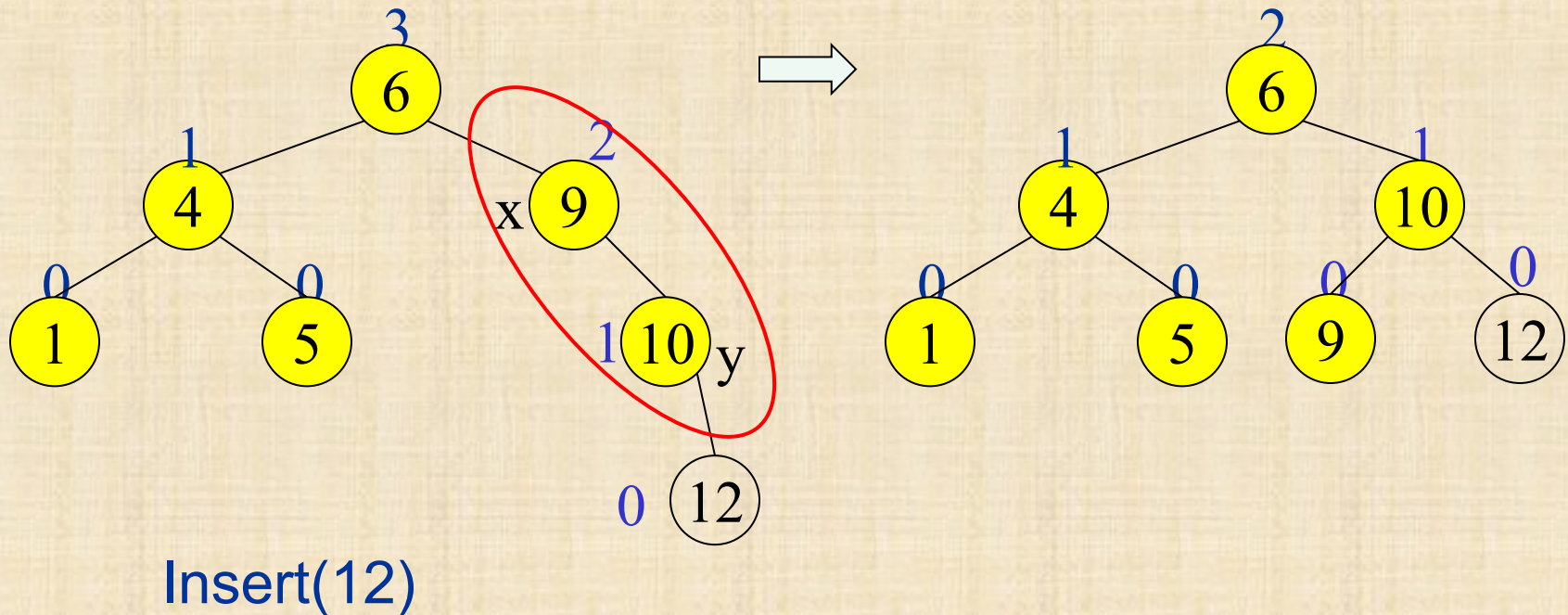


Case 1: y is right-heavy

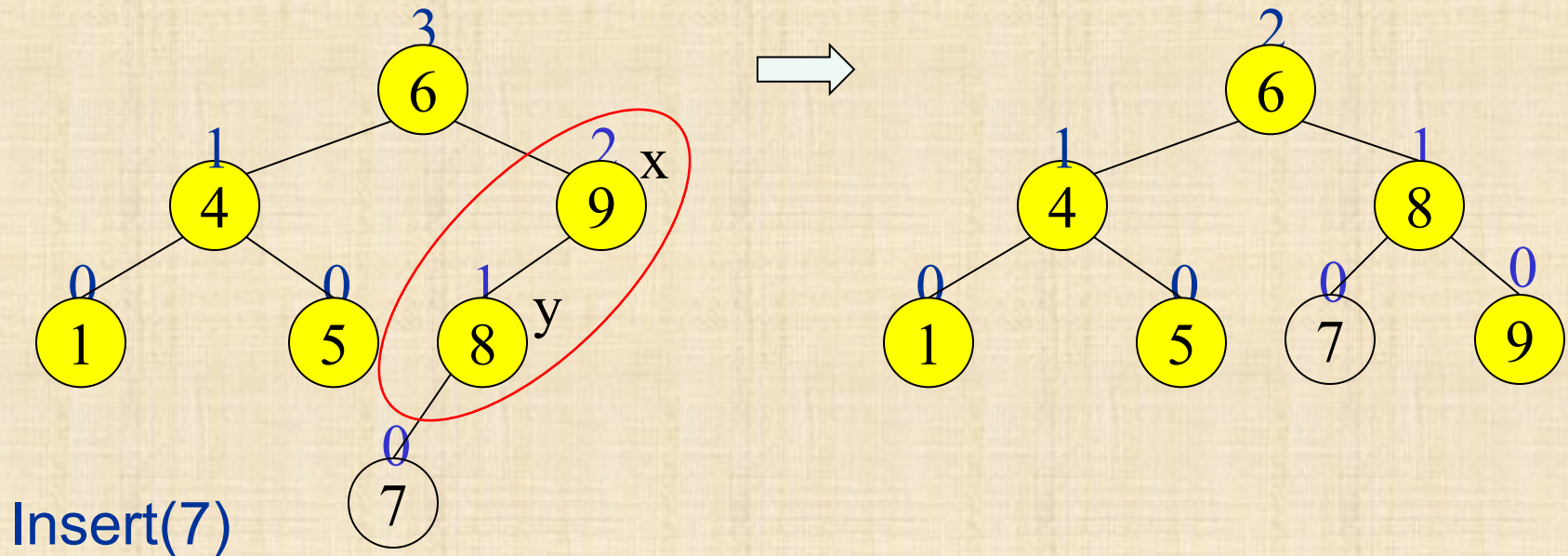


Rebalance by performing a single left rotation

Example: Single Left Rotation in an AVL Tree

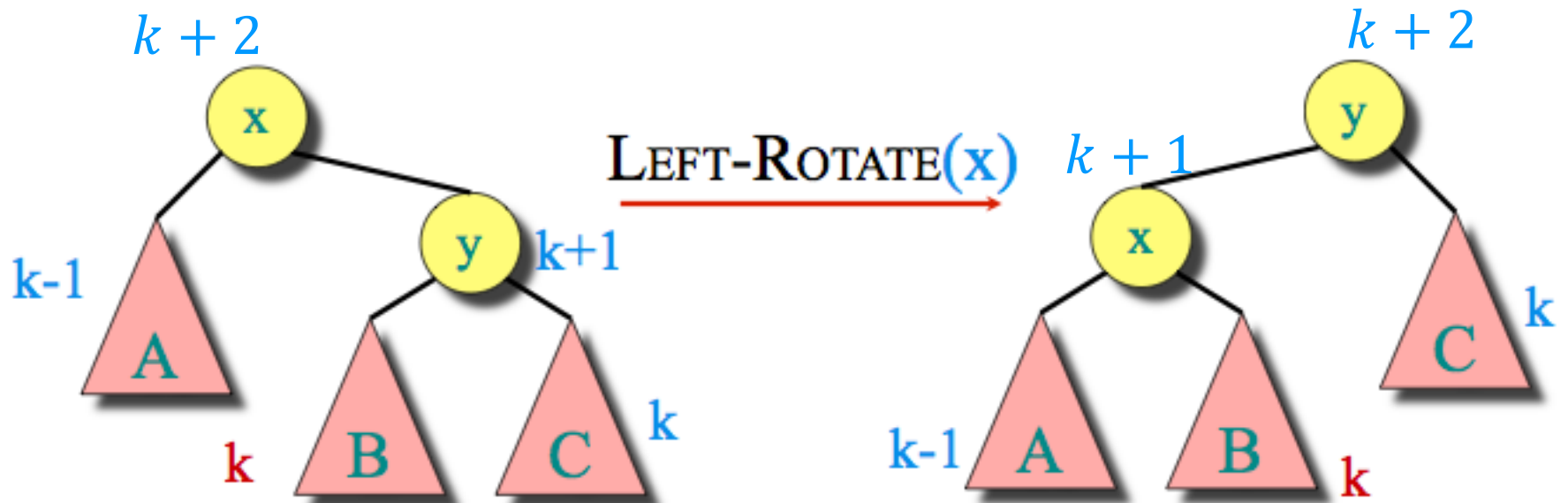


Example: Single Right Rotation in an AVL Tree



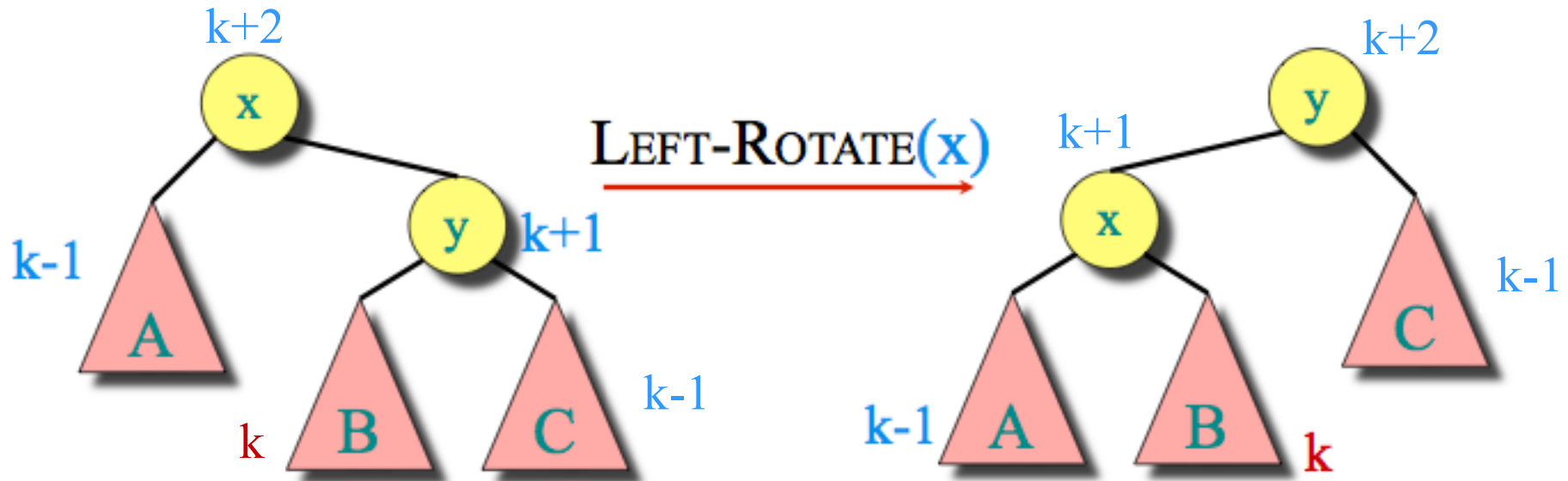
The symmetric case (x is left-heavy and y is left-heavy):
Perform a single right rotation.

Case 1: y is balanced



Same as y is right heavy

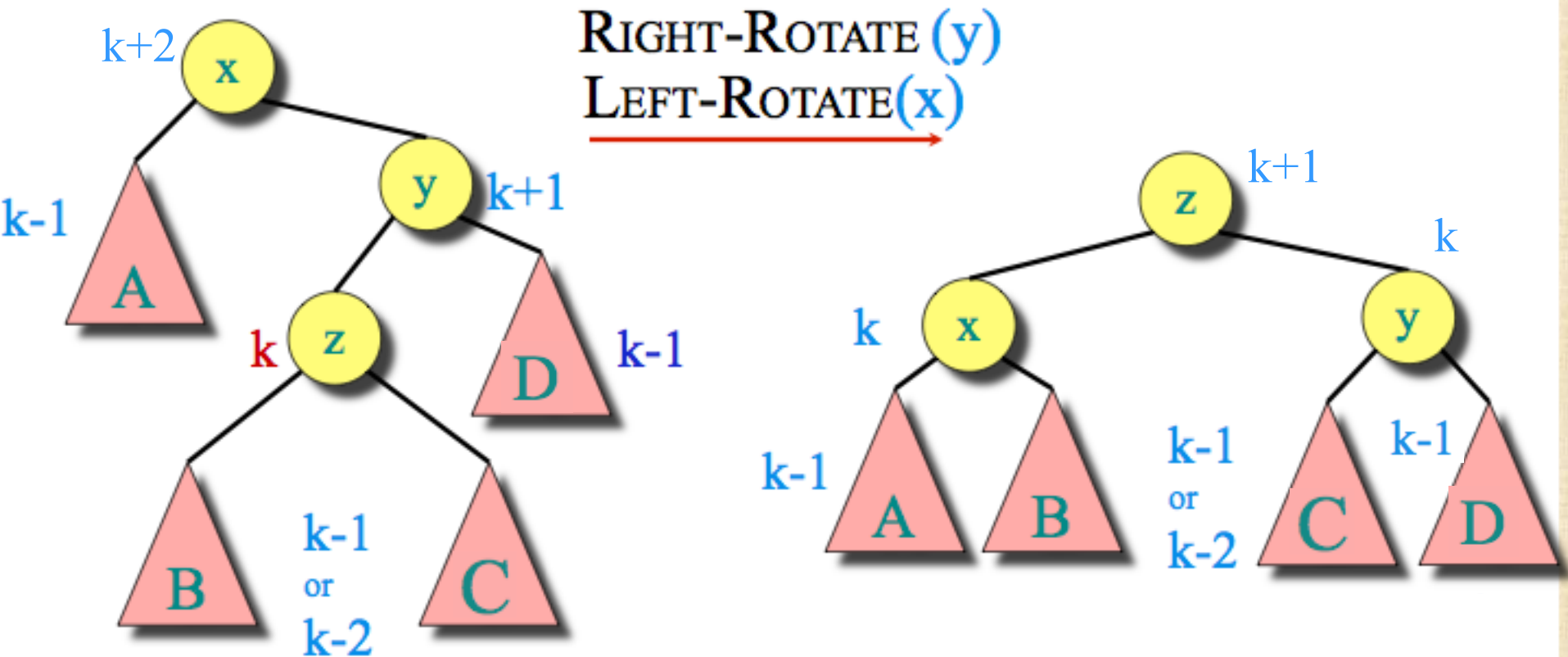
Case 2: y is left-heavy



A single rotation is not sufficient,
because now y is not balanced.
We need something else...



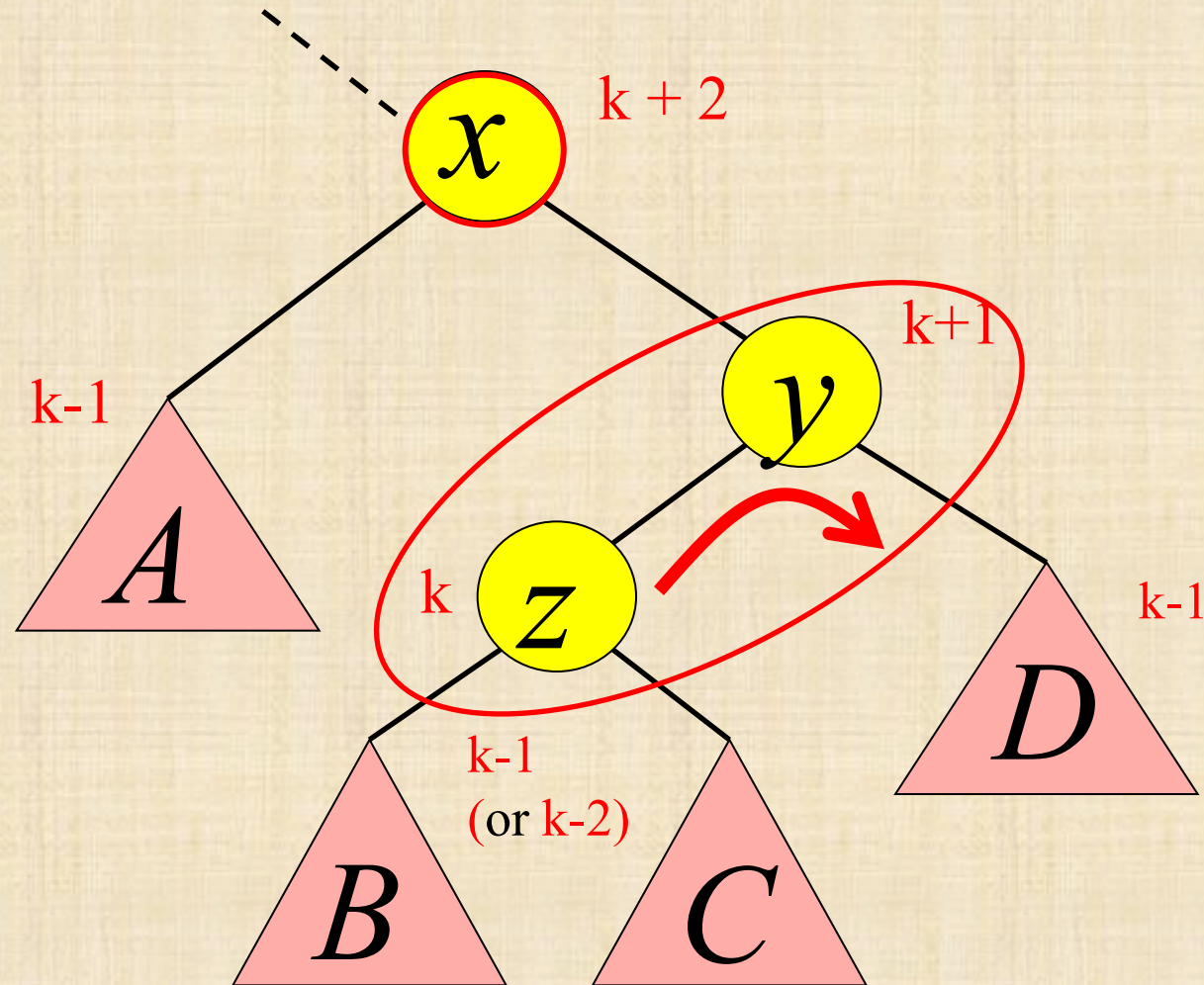
Case 3: y is left-heavy



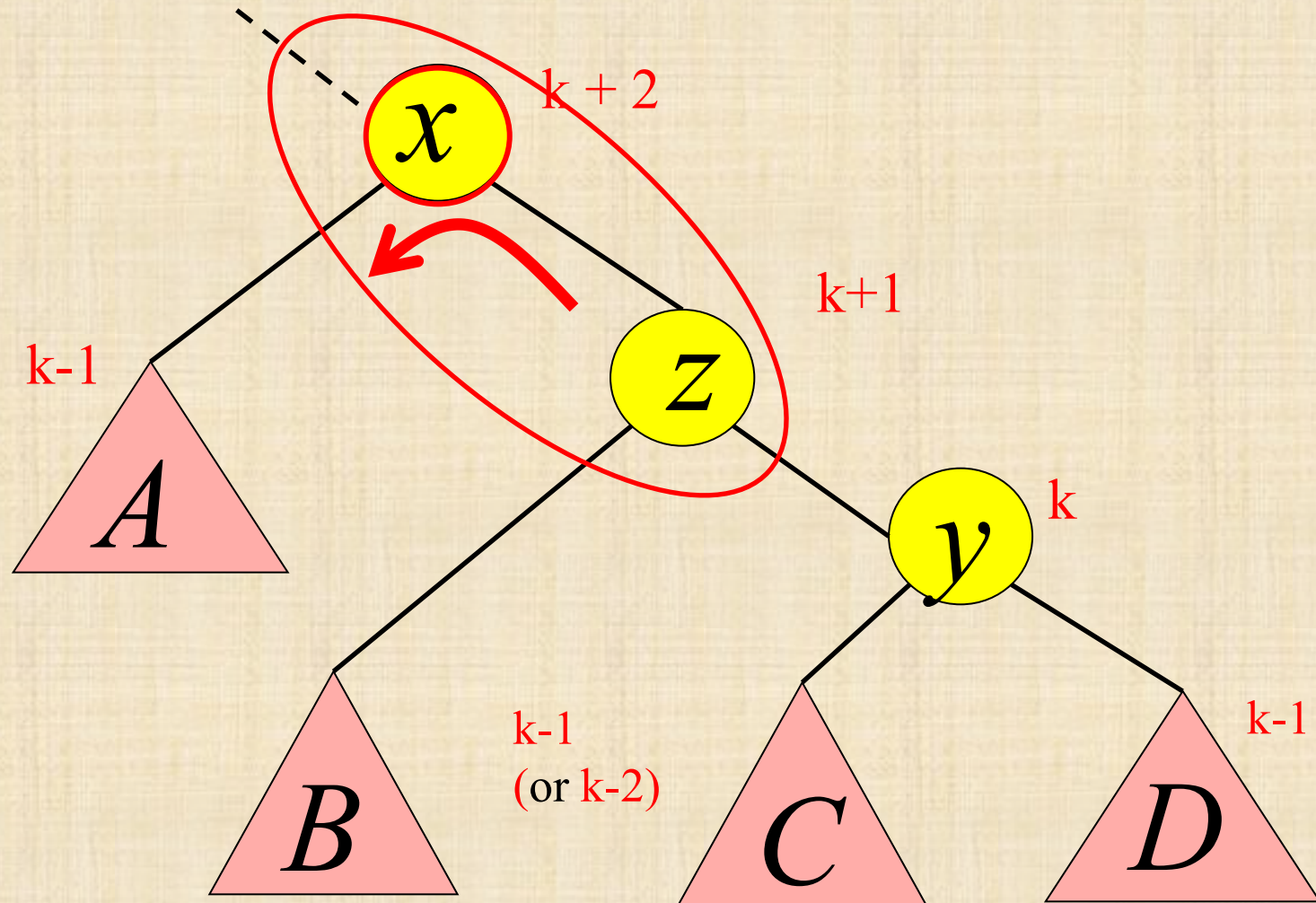
Balance restored.



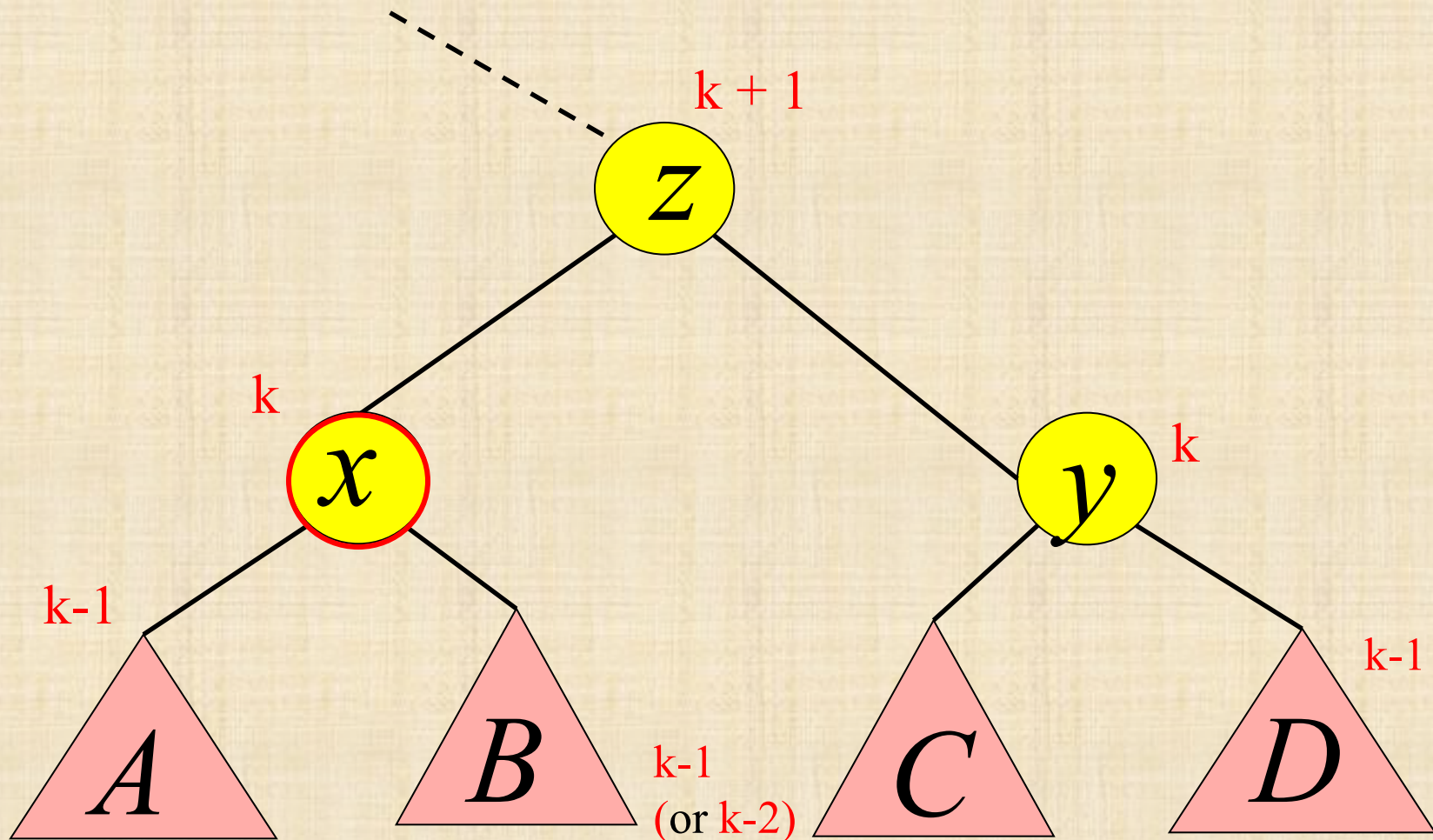
Double rotation



Double rotation



Double rotation



Summary: Insertion in AVL Trees

- ▷ Insert at the leaf (as for all BST)
 - ▷ Only nodes on the path from insertion point to the root might change their height
 - ▷ After the Insertion, go up to the root node by node, updating the heights (can be performed as part of the recursion)
 - ▷ If for some node ($h_{\text{left}} - h_{\text{right}}$) became 2 or -2, adjust tree by a *rotation* or a *double rotation* around that node


```

INSERT( $x, z$ )
1  if  $x = nil$ 
2      then  $height[z] \leftarrow 0$ 
3          return  $z$ 
4  if  $key[z] \leq key[x]$ 
5      then  $y \leftarrow INSERT(left[x], z)$ 
6           $left[x] \leftarrow y$ 
7           $parent[y] \leftarrow x$ 
8           $height[x] \leftarrow height[y] + 1$ 
9  else  $y \leftarrow INSERT(right[x], z)$ 
10      $right[x] \leftarrow y$ 
11      $parent[y] \leftarrow x$ 
12      $height[x] \leftarrow height[y] + 1$ 
13  $x \leftarrow BALANCE(x)$ 
14 return  $x$ 

```

```

BALANCE( $x$ )
1  if  $|height(left[x]) - height(right[x])| \leq 1$ 
2      then return  $x$ 
3  elseif  $height(left[x]) > height(right[x])$ 
4      then  $y \leftarrow left[x]$ 
5          if  $height(left[y]) < height(right[y])$ 
6              then LEFT-ROTATE( $y$ )
7              return RIGHT-ROTATE( $x$ )
8  else  $y \leftarrow right[x]$ 
9      if  $height(left[y]) > height(right[y])$ 
10          then RIGHT-ROTATE( $y$ )
11          return LEFT-ROTATE( $x$ )

```

AVL Tree Deletion

▷ Two options:

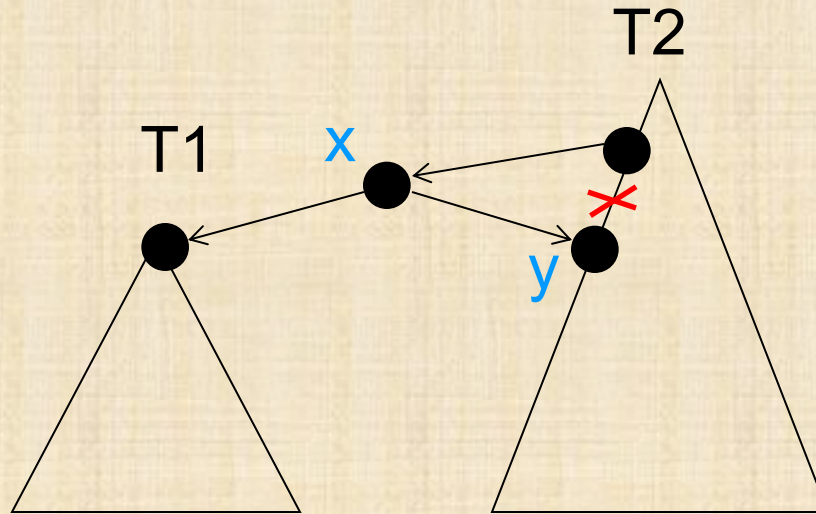
1. Similar handling to insertion

- ▷ Rotations and double rotations to fix balance along the path to the root
- ▷ Imbalance may propagate upward so that many rotations may be needed
- ▷ A bit more complicated than insertion

Or

2. Use Split and Join

Join(T1,x,T2)

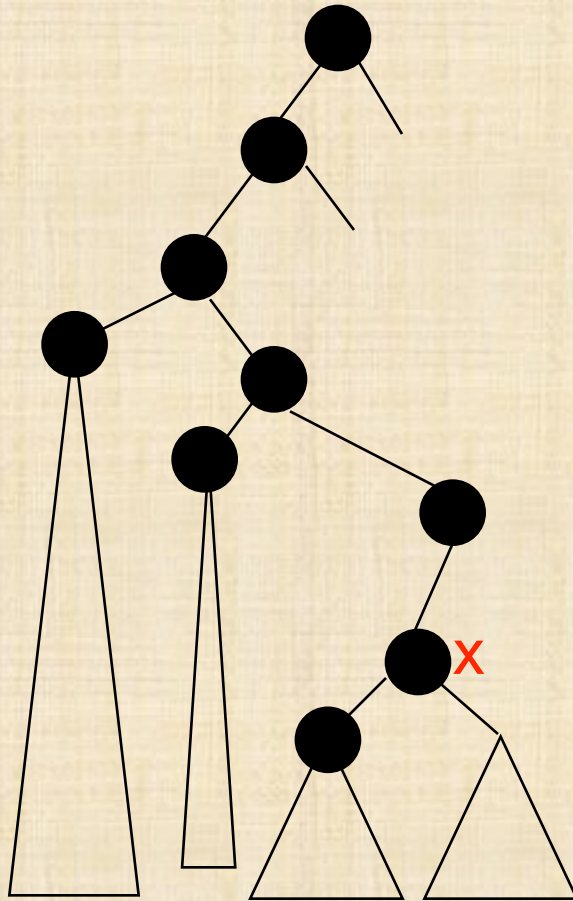


Assume **T2** is higher than **T1** (the other case is symmetric)
Search the left spine of **T2** for a node **y** of the same height
as the root of **T1**

Make **x** a left child of **parent(y)**. Make **y** a right child of **x**.
Continue rotations towards the root as for insert.

$O(|h_2 - h_1| + 1) = O(\log n)$ worst case.

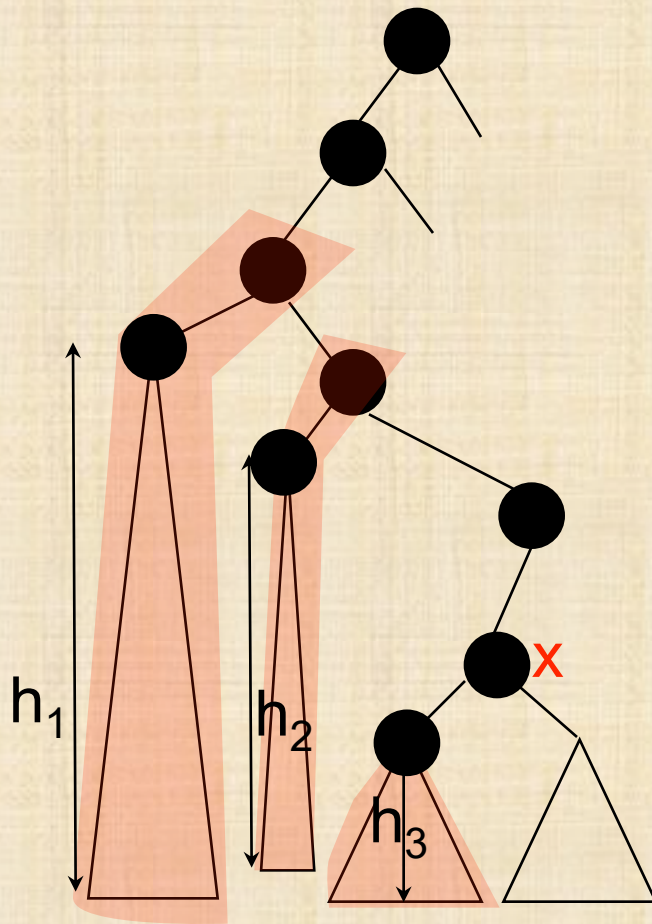
Split(T, x) into $T_1 + x + T_2$



Join all trees to the left of the path from the root to x to form the left tree T_1 .

Join all trees to the right of the path from the root to x to form the right tree T_2

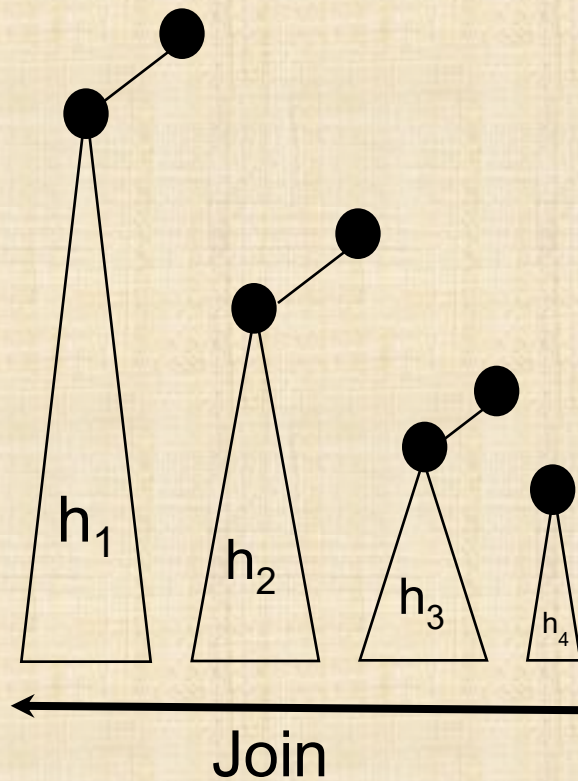
Split(T, x) into $T_1 + x + T_2$



$$h_1 \geq h_2 \geq h_3$$

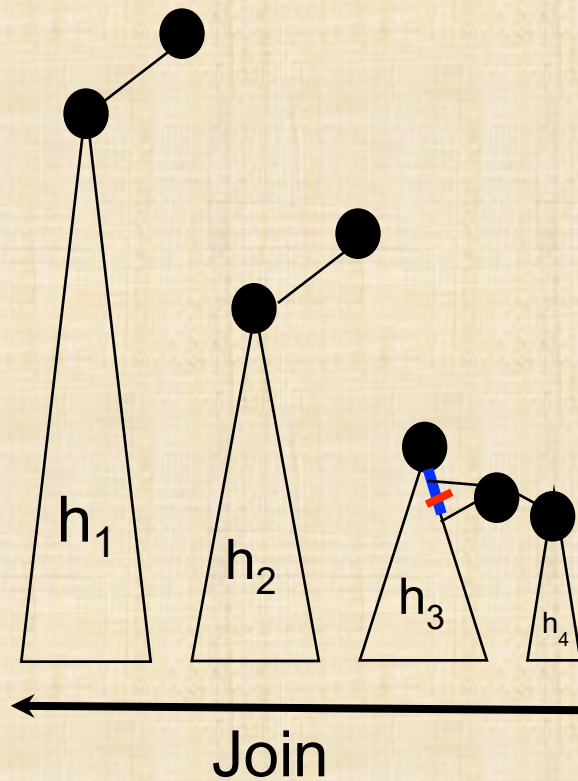
Join all trees to the left of the path from the root to x to the left tree T_1 (including x).

Split(T,x) means joining logn trees



$$h_1 \geq h_2 \geq h_3 \geq h_4$$

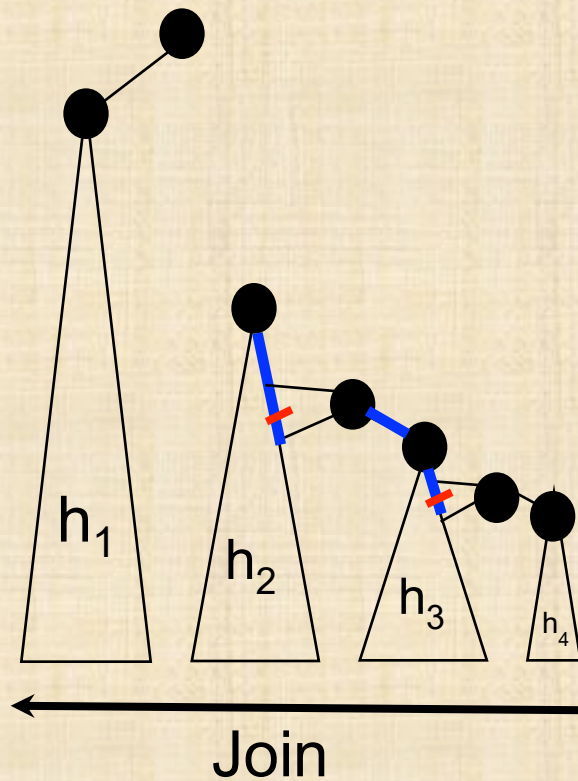
Split(T,x) means joining logn trees



$$h_1 \geq h_2 \geq h_3 \geq h_4$$

$$\text{Time} = O(1) + O(h_3 - h_4) \leq c(1 + h_3 - h_4)$$

Split(T,x) means joining logn trees

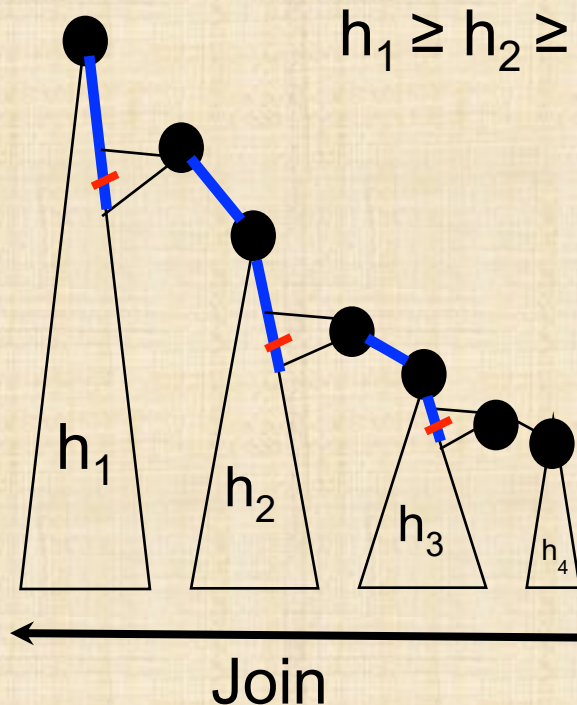


$$h_1 \geq h_2 \geq h_3 \geq h_4$$

Time $\leq$

$$c(1 + h_3 - h_4) \\ + c(1 + h_2 - h_3)$$

Split(T,x) means joining *log*n trees



Time $\leq$

$$c(1 + \cancel{h_3} - h_4)$$

$$+ c(1 + \cancel{h_2} - \cancel{h_3})$$

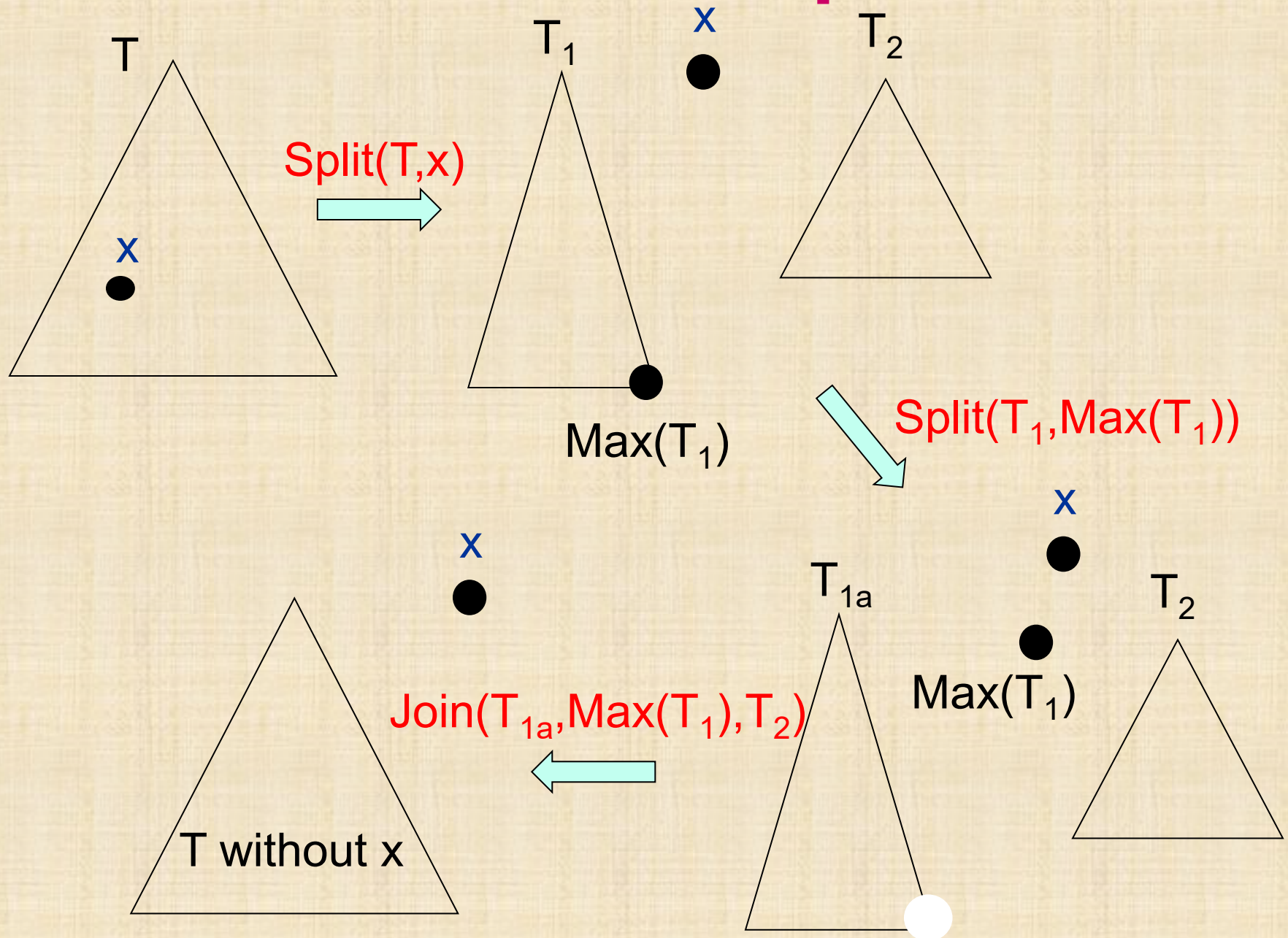
$$+ c(1 + h_1 - \cancel{h_2})$$

$$\leq c(\log n + h_1 - h_4)$$

$$= O(\log n)$$

This sequence of joins handles elements $< x$.
Another $O(\log n)$ is required for elements $> x$

AVL deletion with Split and Join



AVL deletion with Split and Join

▷ Delete(T, x):

▷ $T_1, x, T_2 = \text{Split}(T, x)$

▷ $T_{1a}, y, T_{1b} = \text{Split}(T_1, \text{Max}(T_1))$

Note: T_{1b} is empty

▷ return Join(T_{1a}, y, T_2)

Augmenting AVL-Trees

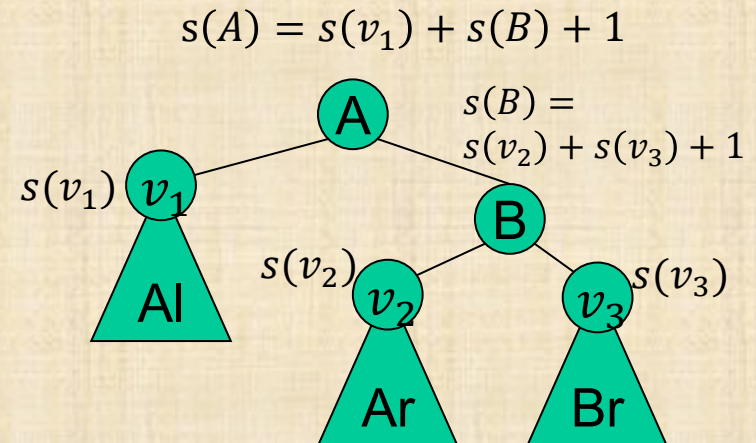
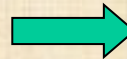
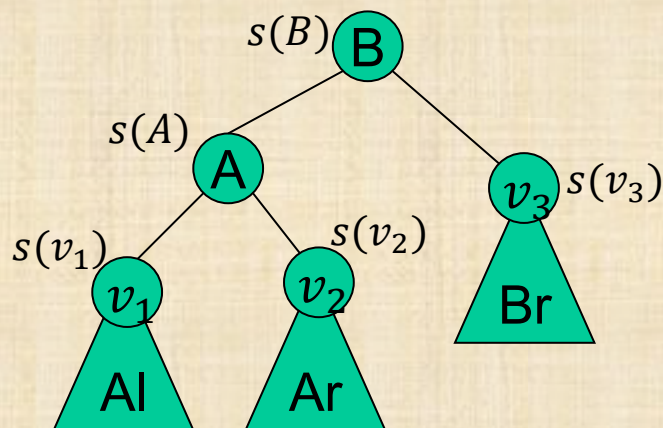
Theorem

Theorem: Let f be a field that augments an AVL-tree. If the value of f for each node x can be computed using only the information in x , $x.\text{left}$, $x.\text{right}$, $x.\text{left}.f$ and $x.\text{right}.f$, then it is possible to maintain the values of f in all nodes during insertions and deletions in $O(\log n)$ time.

Example

Update $size(v)$

1. When inserting a node, increase by 1 all the values $size(v)$ along the insertion path from the root to the leaf. The new node gets $size(x)=1$
2. During rotations, update the values as needed locally: example:
Left rotation



Additional $O(1)$ operations per rotation

AVL Tree: demo

Starting from an empty tree, insert the keys
10, 20, 30, 25, 27, 26, 40
Now delete 20

<https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>

Pros of AVL Trees

- ▷ height is $O(\log n)$ since AVL trees are always balanced
- ▷ so all operations we have seen take $O(\log n)$ time
- ▷ Maintaining the balance can be done within the same $O(\log n)$ time

Cons of AVL Trees

1. Difficult to program & debug
2. More space (height augmentation)
3. Rebalancing costs time (larger constants)

Summary so far

- ▷ Binary Search Trees
 - ▷ basic
 - ▷ augmented
 - ▷ balanced (AVL)
- ▷ next up:
 - ▷ another ADT: Disjoint Sets / Union-Find

Disjoint Sets / Union-Find

A data structure that maintains a set of elements partitioned into disjoint sets

Recall: Two sets are **disjoint** if they have no elements in common

Motivating Examples

- ▷ A network of computers, some pairs of computers are connected by a communication link
 - ▷ New links can be generated
 - ▷ Test whether two computers are connected, not necessarily directly
- ▷ A social network of people, some pairs are friends
 - ▷ Two people are connected if there is a chain of friends from one to the other
 - ▷ New friendship may be generated
 - ▷ Test whether two people are connected

ADT: Union Find

- ▷ A universe S of elements is partitioned into disjoint sets
- ▷ Each set has a leader (one of its members)

Operations:

- ▷ **MakeSet(x)**: creates a new set whose only member (and thus leader) is x
- ▷ **Find(x)**: Find the set (the leader) to which x belongs
- ▷ **Union(a,b)**: Unite the two sets whose leaders are a and b and choose a new leader

Disjoint Sets - Example

- ▷ Let $S = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- ▷ A possible partition into disjoint sets:
 $\{3, 5, 7\}$, $\{4, 2, 8\}$, $\{9\}$, $\{1, 6\}$
- ▷ Each set has a unique leader, one of its members
 $\{3, \underline{5}, 7\}$, $\{4, 2, \underline{8}\}$, $\{\underline{9}\}$, $\{\underline{1}, 6\}$
- ▷ The leader is used as the name of the set.

Find

- ▷ *Find*(z) – return the name of the set containing z
 - ▷ $\{3, \underline{5}, 7, 1, 6\}, \{4, 2, \underline{8}\}, \{\underline{9}\}$
 - ▷ $\text{Find}(1) = 5$
 - ▷ $\text{Find}(4) = 8$
 - ▷ $\text{Find}(8) = 8$

Union

Union(x,y):

- ▷ merge the two sets whose leaders are x and y
- ▷ Choose a leader for the resulting set (usually x or y)
- ▷ Example:
 - ▷ $\{3, \underline{5}, 7\}$, $\{4, 2, \underline{8}\}$, $\{\underline{9}\}$, $\{\underline{1}, 6\}$
 - ▷ $\text{Union}(5, 1)$
 $\{3, \underline{5}, 7, 1, 6\}$, $\{4, 2, \underline{8}\}$, $\{\underline{9}\}$

An undirected Graph: $G=(V,E)$

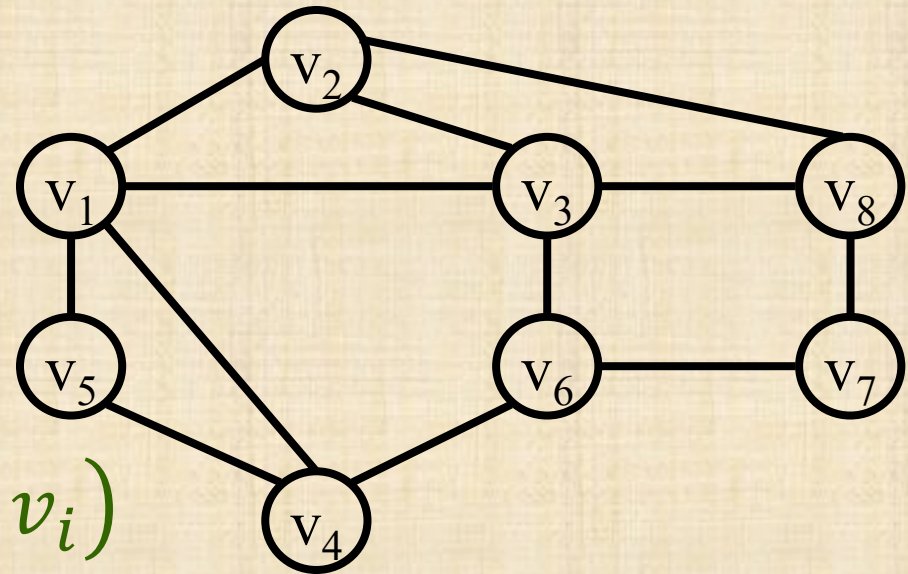
- ▷ Set of vertices : $V = \{v_1, v_2, \dots, v_n\}$
- ▷ Set of edges: $E = \{\{v_1, v_2\}, \{v_1, v_3\}, \dots, \{v_7, v_8\}\}$

▷ $v_i \in V$

▷ Convention:

Write (v_i, v_j)

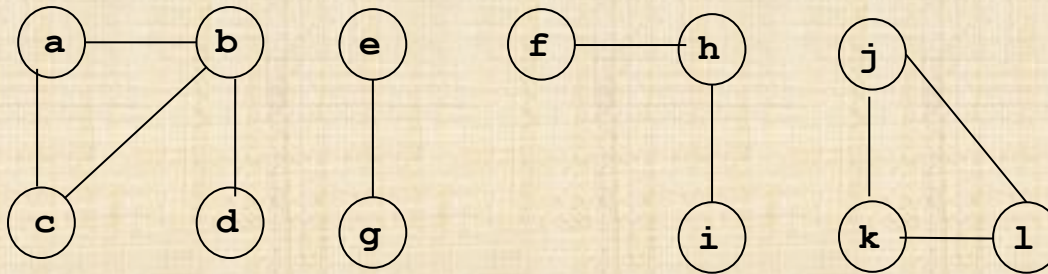
mean $(v_i, v_j) = (v_j, v_i)$



Connected Component

Definition: a connected component is a maximal set of vertices in which any two vertices are connected by a path

not included in a larger such set



A graph with 4 connected components:

$\{a, b, c, d\}, \{e, g\}, \{f, h, i\}, \{j, k, l\}$

Note: Our social-network problem is to find the connected components of the graph describing the network

Maintenance of Connected Components

Let V denote the set of people

Let E denote the set of friendship connections

1. Create a singleton set for each person $v \in V$

2. For each pair $(u, v) \in E$

if $Find(u) \neq Find(v)$ then

$Union(Find(u), Find(v))$

Note: vertices i and j are connected *if and only if*
 $Find(i) = Find(j)$

Connected-Component(G)

for each vertex $v \in V(G)$

do *MakeSet*(v)

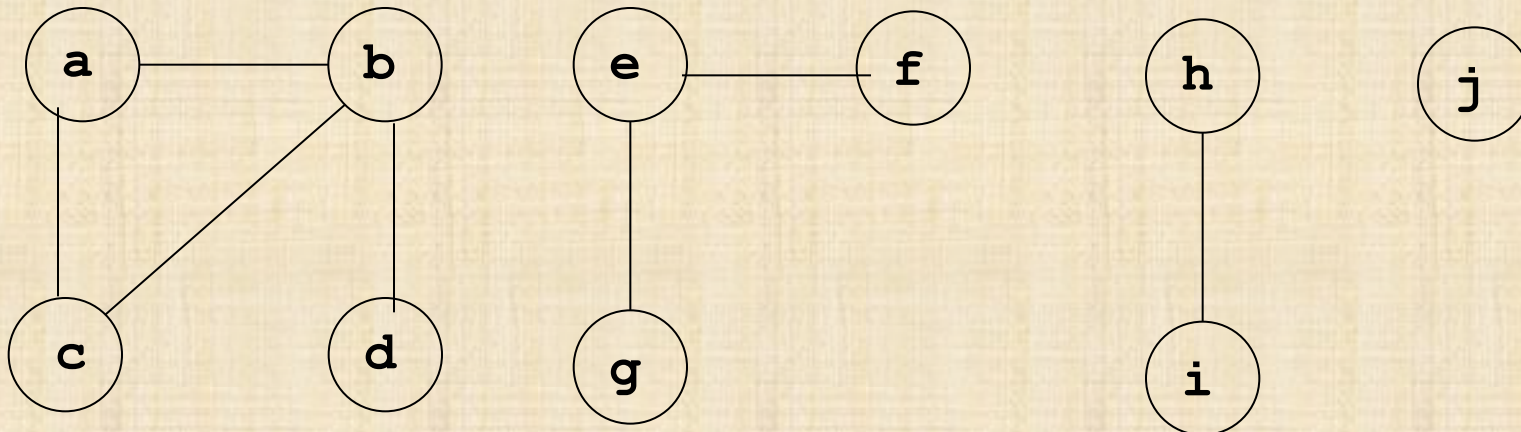
for each edge $(u,v) \in E(G)$

do if $\text{Find}(u) \neq \text{Find}(v)$

then $\text{Union}(\text{Find}(u), \text{Find}(v))$

$V = \{a, b, c, d, e, f, g, h, i, j\}$

$E = \{(a, c), (g, e), (i, h), (e, f),$
 $(b, d), (a, b), (c, b)\}$



Connected-Component(G)

for each vertex $v \in V(G)$

do *MakeSet(v)*

for each edge $(u,v) \in E(G)$

do if $\text{Find}(u) \neq \text{Find}(v)$

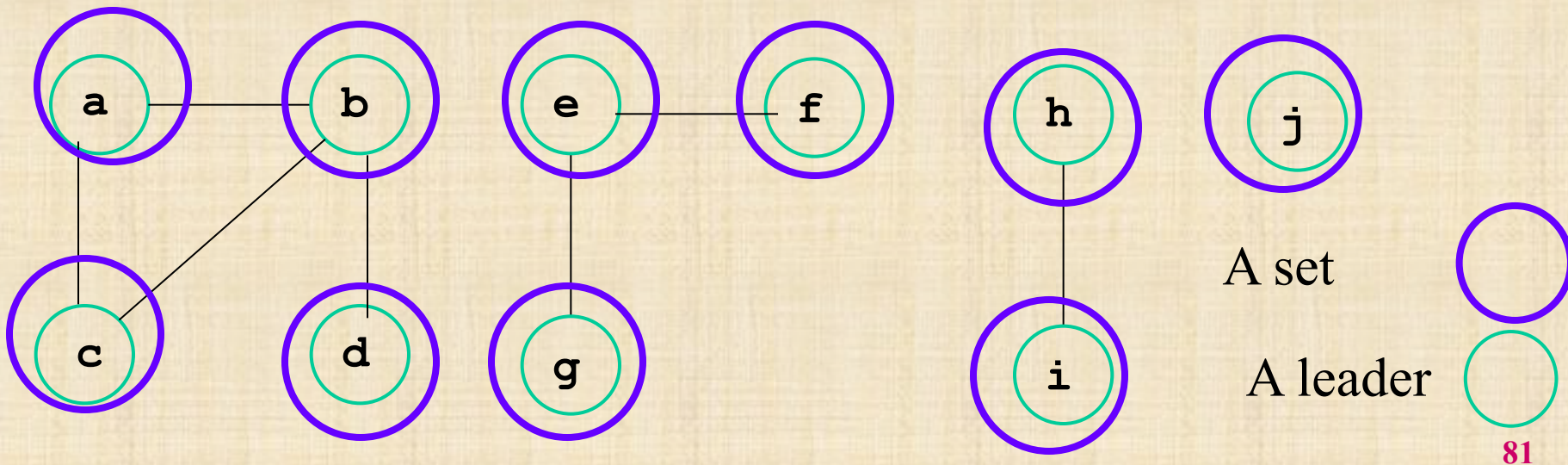
then $\text{Union}(\text{Find}(u), \text{Find}(v))$

$V = \{a, b, c, d, e, f, g, h, i, j\}$

$E = \{(a, c), (g, e), (i, h), (e, f),$
 $(b, d), (a, b), (c, b)\}$

Initially each set contains a single vertex:

$\{\{a\}, \{b\}, \{c\}, \{d\}, \{e\}, \{f\}, \{g\}, \{h\}, \{i\}, \{j\}\}$



Connected-Component(G)

for each vertex $v \in V(G)$

do *MakeSet*(v)

for each edge $(u,v) \in E(G)$

do if $\text{Find}(u) \neq \text{Find}(v)$

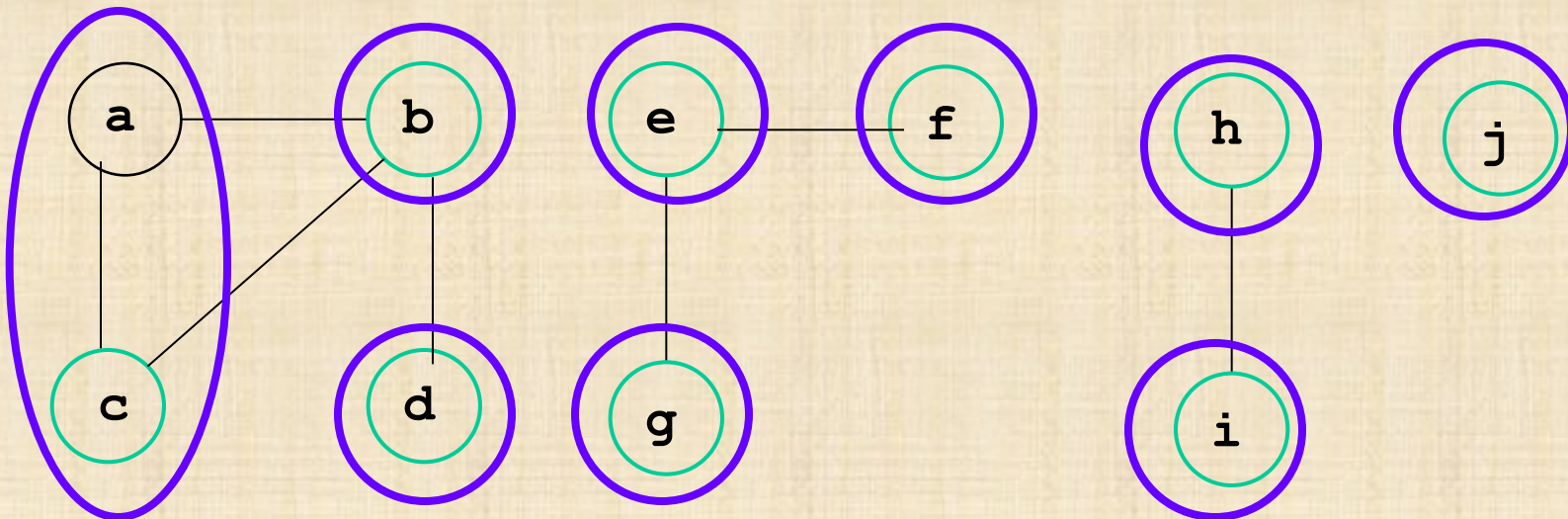
then *Union*($\text{Find}(u), \text{Find}(v)$)

$V = \{a, b, c, d, e, f, g, h, i, j\}$

$E = \{(a,c), (g,e), (i,h), (e,f),$
 $(b,d), (a,b), (c,b)\}$

$\{\{a\}, \{b\}, \{c\}, \{d\}, \{e\}, \{f\}, \{g\}, \{h\}, \{i\}, \{j\}\}$

$\{\{a,c\}, \{b\}, \{d\}, \{e\}, \{f\}, \{g\}, \{h\}, \{i\}, \{j\}\}$



Connected-Component(G)

for each vertex $v \in V(G)$

do *MakeSet*(v)

for each edge $(u,v) \in E(G)$

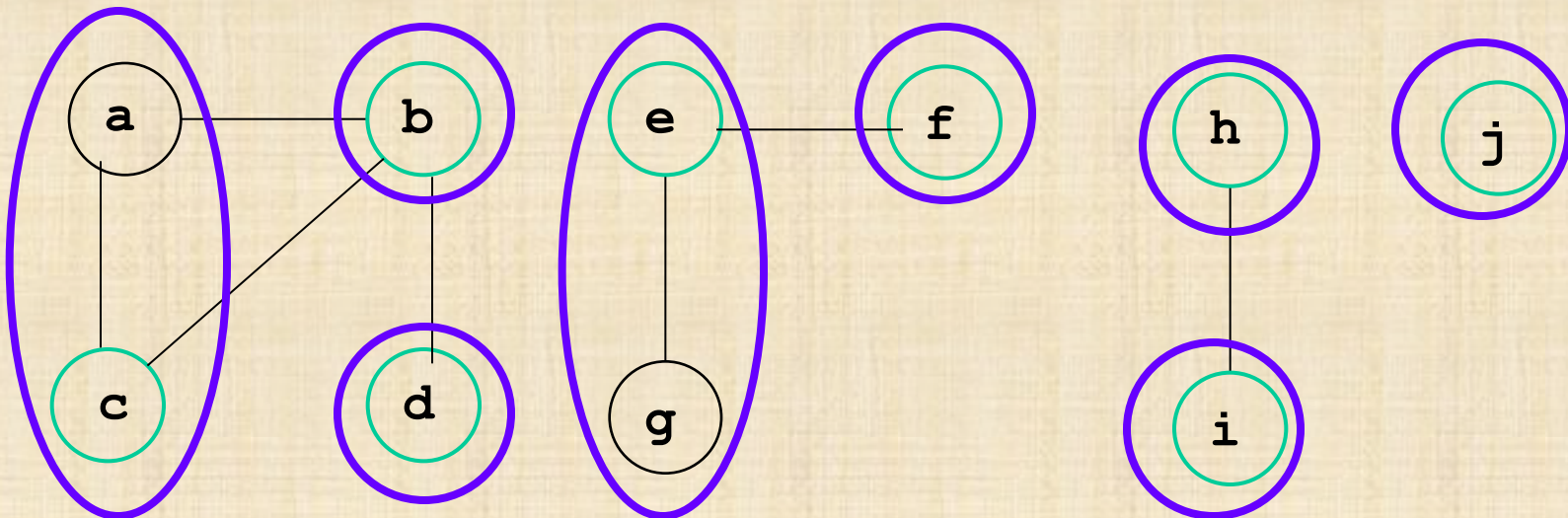
do if $\text{Find}(u) \neq \text{Find}(v)$

then *Union*($\text{Find}(u), \text{Find}(v)$)

$V = \{a, b, c, d, e, f, g, h, i, j\}$

$E = \{(a, c), (g, e), (i, h), (e, f),$
 $(b, d), (a, b), (c, b)\}$

$\{\{a, c\}, \{b\}, \{d\}, \{e\}, \{f\}, \{g\}, \{h\}, \{i\}, \{j\}\}$
 $\{\{a, c\}, \{b\}, \{d\}, \{e, g\}, \{f\}, \{h\}, \{i\}, \{j\}\}$



Connected-Component(G)

for each vertex $v \in V(G)$

do *MakeSet*(v)

for each edge $(u,v) \in E(G)$

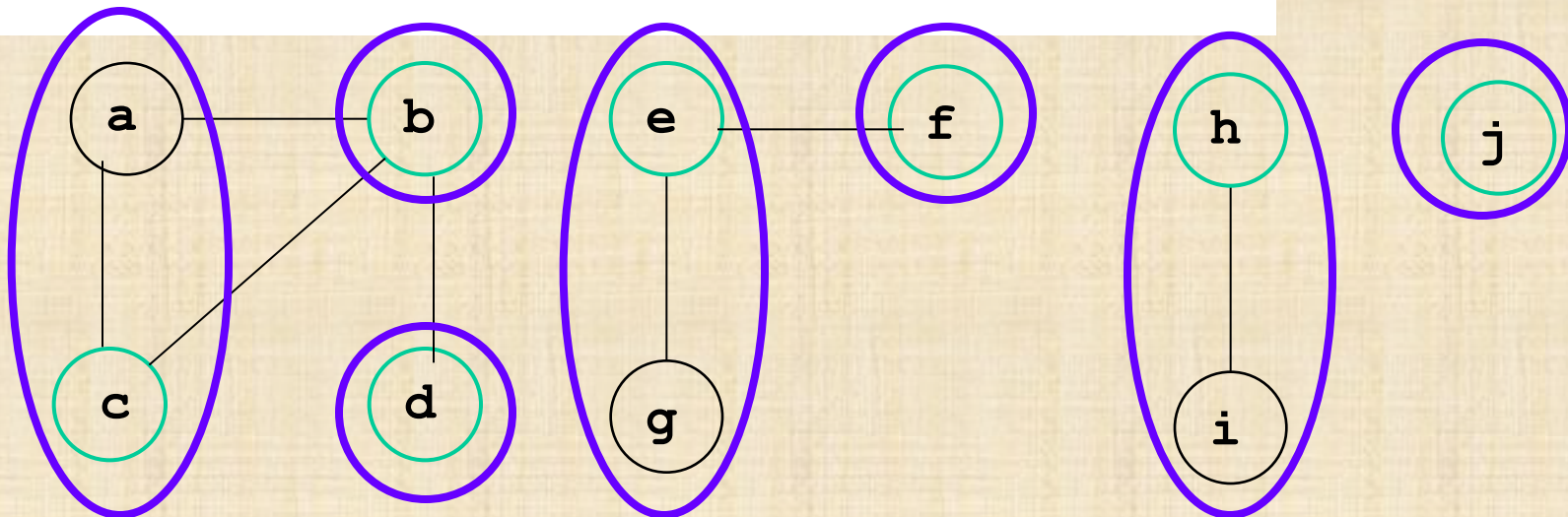
do if $\text{Find}(u) \neq \text{Find}(v)$

then *Union*($\text{Find}(u), \text{Find}(v)$)

$V = \{a, b, c, d, e, f, g, h, i, j\}$

$E = \{(a,c), (g,e), (i,h), (e,f),$
 $(b,d), (a,b), (c,b)\}$

$\{\{a,c\}, \{b\}, \{d\}, \{e,g\}, \{f\}, \{h\}, \{i\}, \{j\}\}$
 $\{\{a,c\}, \{b\}, \{d\}, \{e,g\}, \{f\}, \{i,h\}, \{j\}\}$



Connected-Component(G)

for each vertex $v \in V(G)$

do *MakeSet*(v)

for each edge $(u,v) \in E(G)$

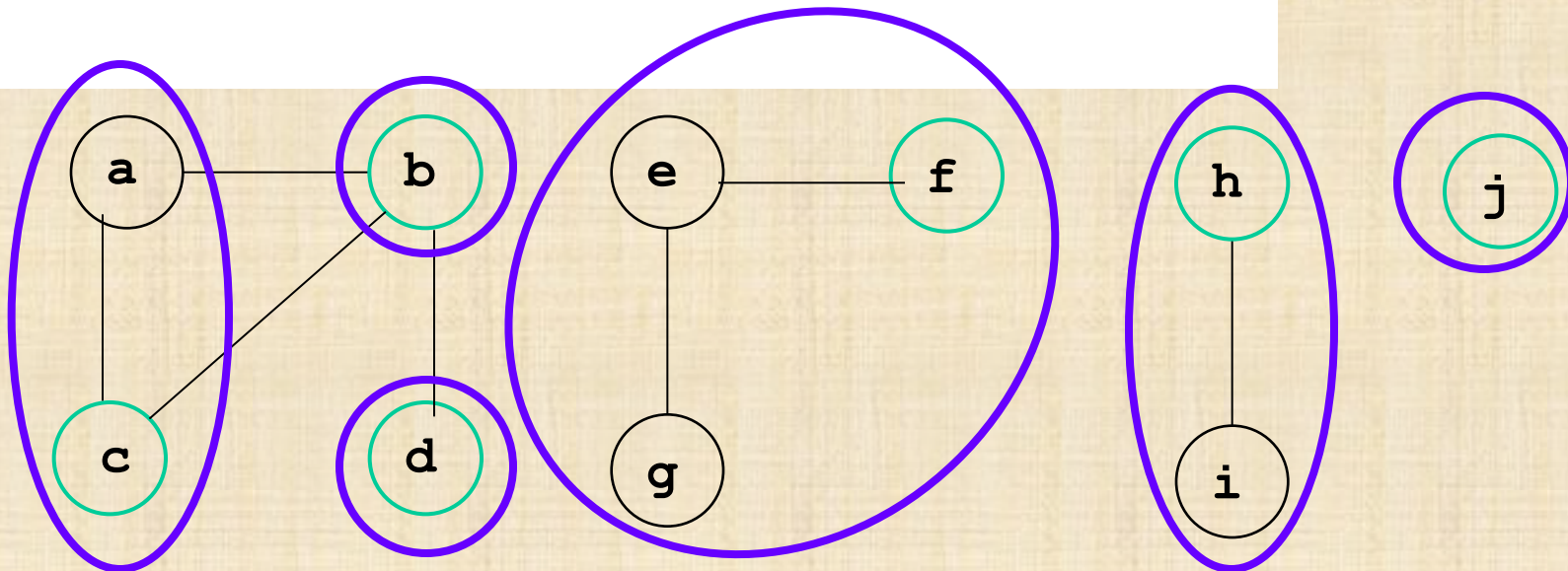
do if $\text{Find}(u) \neq \text{Find}(v)$

then *Union*($\text{Find}(u), \text{Find}(v)$)

$V = \{a, b, c, d, e, f, g, h, i, j\}$

$E = \{(a, c), (g, e), (i, h), (e, f),$
 $(b, d), (a, b), (c, b)\}$

$\{\{a, c\}, \{b\}, \{d\}, \{e, g\}, \{f\}, \{h\}, \{i\}, \{j\}\}$
 $\{\{a, c\}, \{b\}, \{d\}, \{e, g, f\}, \{i, h\}, \{j\}\}$



Connected-Component(G)

for each vertex $v \in V(G)$

do *MakeSet*(v)

for each edge $(u,v) \in E(G)$

do if $\text{Find}(u) \neq \text{Find}(v)$

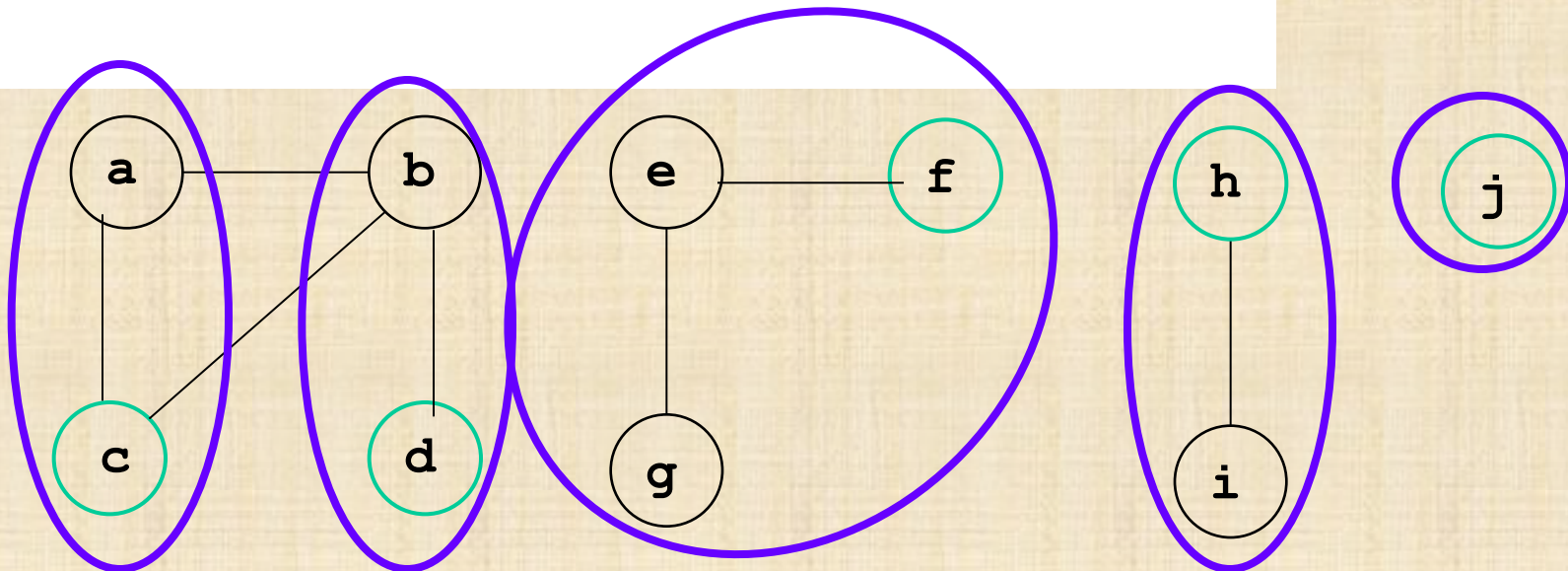
then *Union*($\text{Find}(u), \text{Find}(v)$)

$V = \{a, b, c, d, e, f, g, h, i, j\}$

$E = \{(a, c), (g, e), (i, h), (e, f),$
 $(b, d), (a, b), (c, b)\}$

$\{\{a, c\}, \{b\}, \{d\}, \{e, g, f\}, \{i, h\}, \{j\}\}$

$\{\{a, c\}, \{b, d\}, \{e, g, f\}, \{i, h\}, \{j\}\}$



Connected-Component(G)

for each vertex $v \in V(G)$

do *MakeSet*(v)

for each edge $(u,v) \in E(G)$

do if $\text{Find}(u) \neq \text{Find}(v)$

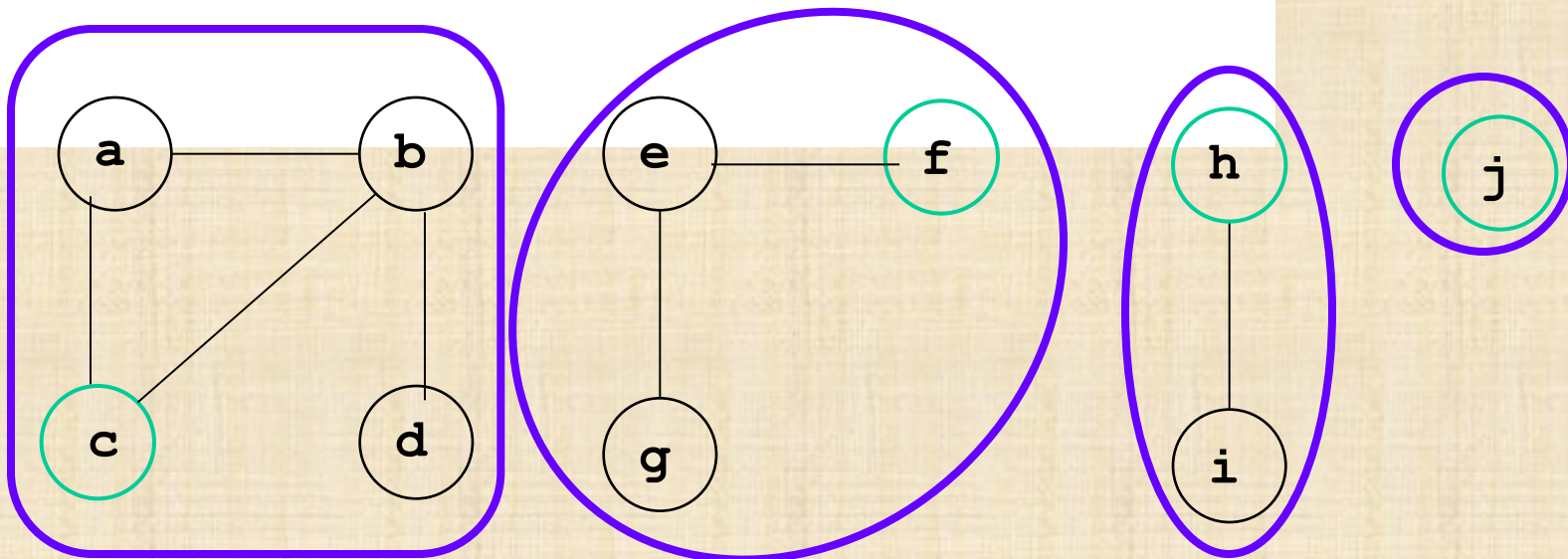
then *Union*($\text{Find}(u), \text{Find}(v)$)

$V = \{a, b, c, d, e, f, g, h, i, j\}$

$E = \{(a, c), (g, e), (i, h), (e, f),$
 $(b, d), (a, b), (c, b)\}$

$\{\{a, c\}, \{b, d\}, \{e, g, f\}, \{i, h\}, \{j\}\}$

$\{\{a, b, c, d\}, \{e, g, f\}, \{i, h\}, \{j\}\}$



Connected-Component(G)

for each vertex $v \in V(G)$

do *MakeSet*(v)

for each edge $(u,v) \in E(G)$

do if $\text{Find}(u) \neq \text{Find}(v)$

then *Union*($\text{Find}(u), \text{Find}(v)$)

$V = \{a, b, c, d, e, f, g, h, i, j\}$

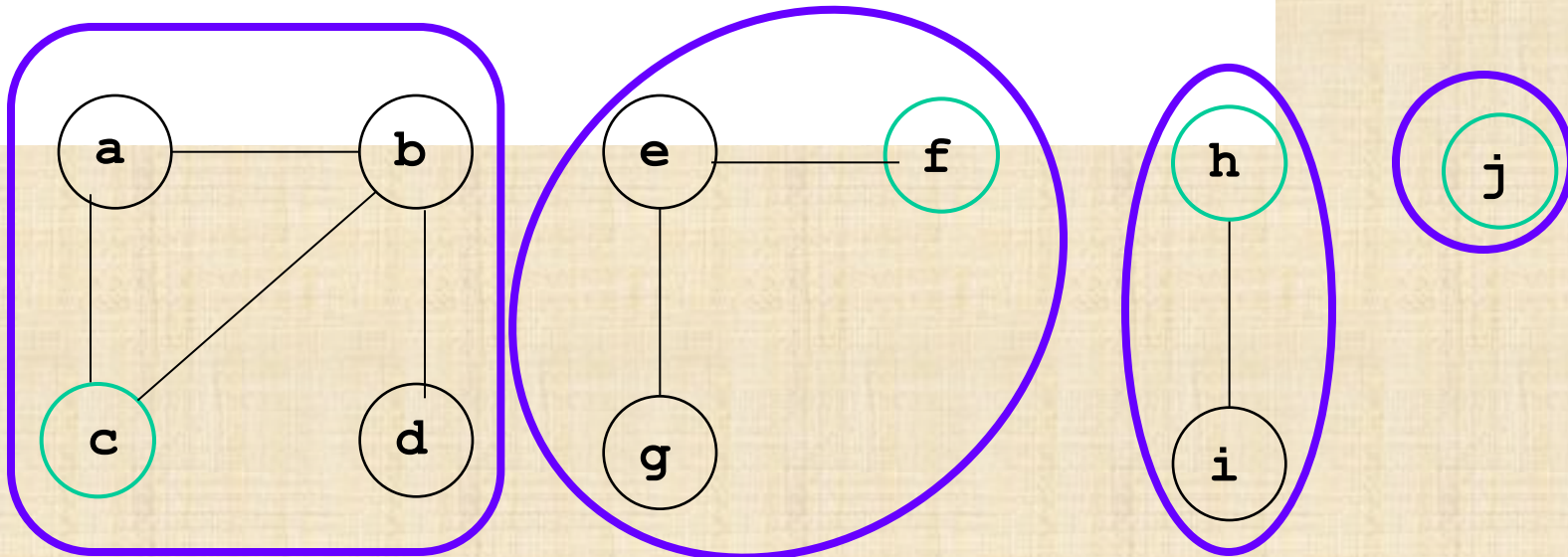
$E = \{(a, c), (g, e), (i, h), (e, f),$
 $(b, d), (a, b), (c, b)\}$

$\{\{a, b, c, d\}, \{e, g, f\}, \{i, h\}, \{j\}\}$

Components detected.

$\text{Find}(c) = \text{Find}(b)$

➔ No union



Implementations of Disjoint Sets

- ▷ Naïve Array
- ▷ A set of linked lists
- ▷ Disjoint-set forest

A Naïve Array

- ▷ Assume that the n elements are named $1, \dots, n$
- ▷ Maintain an array $S[1 \dots n]$, where $S[i]$ stores the leader of the set that element i belongs to

A naïve Array

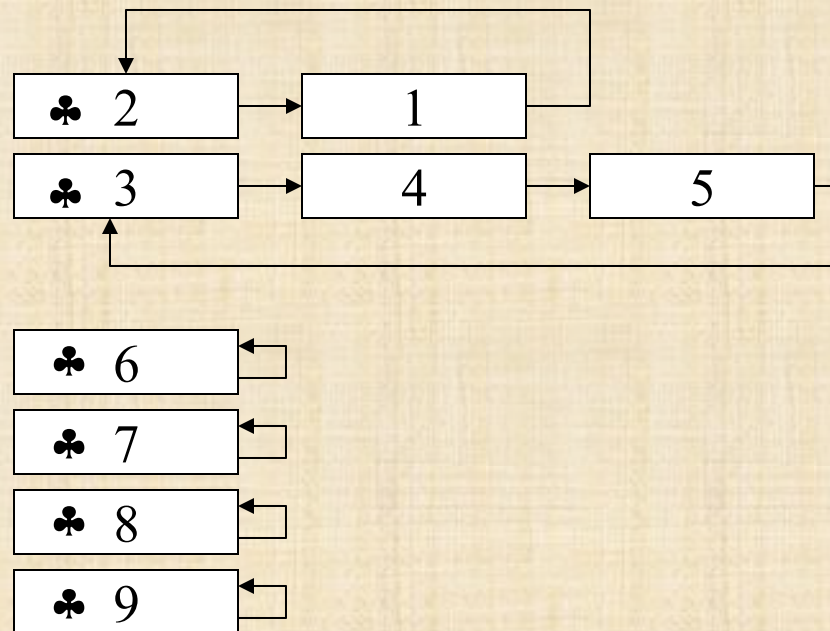
	S - Init state	→ U(2,1)	U(3,4)	U(4,5)	U(2,4)
1	1	2	2	2	4
2	2	2	2	2	4
3	3	3	4	4	4
4	4	4	4	4	4
5	5	5	5	4	4
6	6	6	6	6	6
7	7	7	7	7	7
8	8	8	8	8	8
9	9	9	9	9	9

Time Complexity: *Union*: $\Theta(n)$ *Find*: $\Theta(1)$

Inefficient

A Circular Linked Lists

The elements of every set form a circular linked list.
The leader is marked



Array imp.

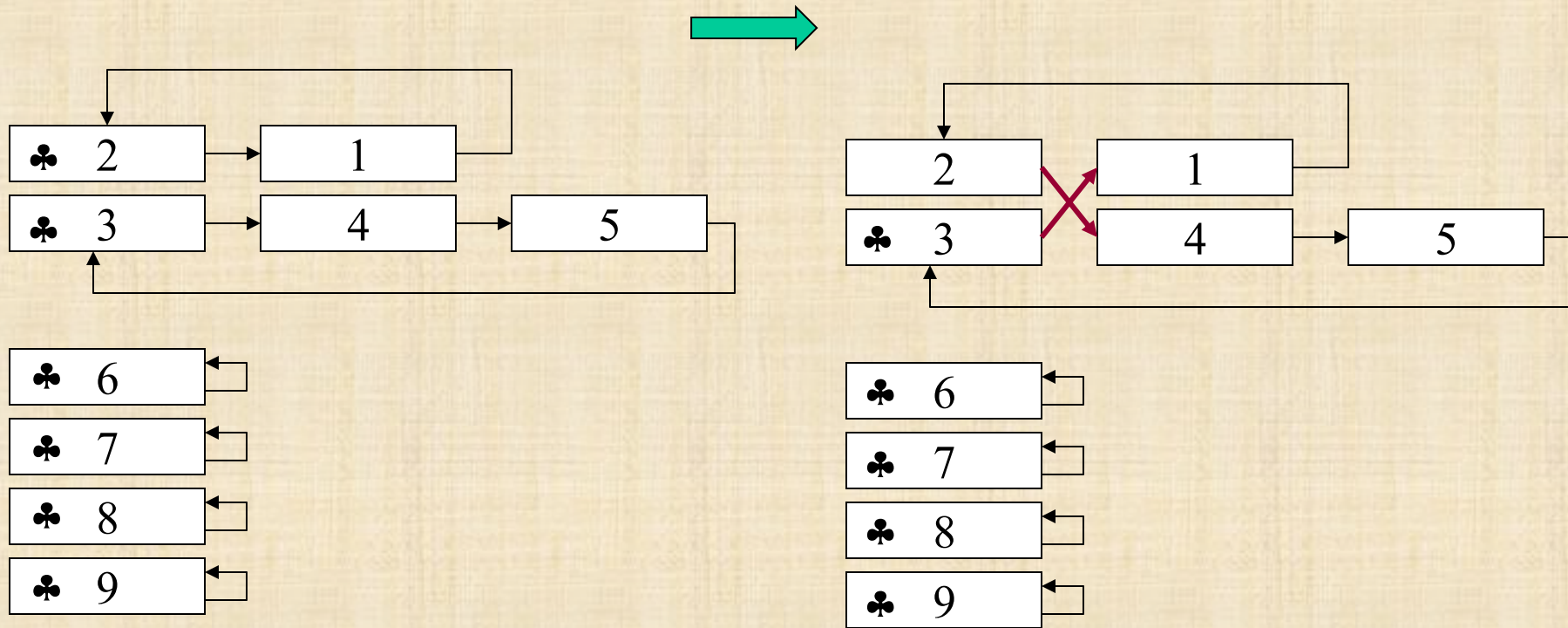
1	2
2	1
3	4
4	5
5	3
6	6
7	7
8	8
9	9

Find(u): advance in u's list till the marked node (the leader)

Union(x,y): Merge the lists of x and y, keep one leader

A Circular Linked Lists

Example: union(2,3)



Time Complexity: *Union:* $\Theta(1)$ *Find:* $\Theta(n)$

Disjoint-Sets Forest

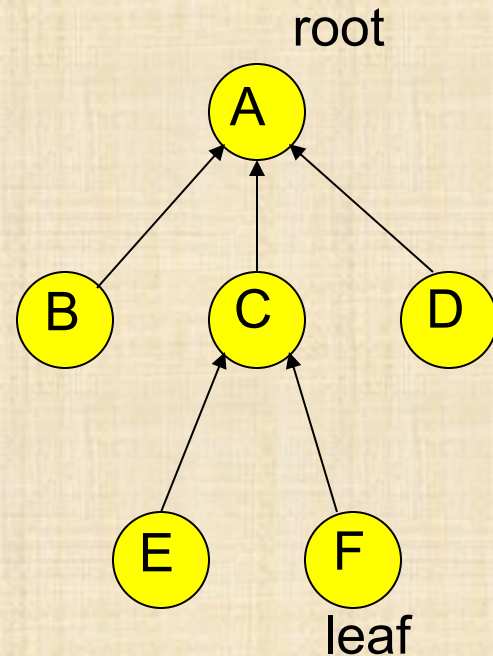
- ▷ We want to do better than
 - ▷ Naïve array: $Union: \Theta(n)$ $Find: \Theta(1)$ or
 - ▷ Lists: $Union: \Theta(1)$ $Find: \Theta(n)$
- ▷ New data structure: *up-tree forest*
 - ▷ Would be relatively easy to get:
 - ▷ $Union: \Theta(1)$
 - ▷ $Find: \Theta(\log n)$
 - ▷ Will be improved further, stay tuned!

Up Tree

- A structure consisting of layers
- A single element in the top layer
- Each element in layer i is pointing to a single element in layer $i-1$

Notations:

- Nodes and edges
- Root
- Path, path length
- **Height of a tree**: the longest path from a leaf to the root
- A **forest** is a set of trees.

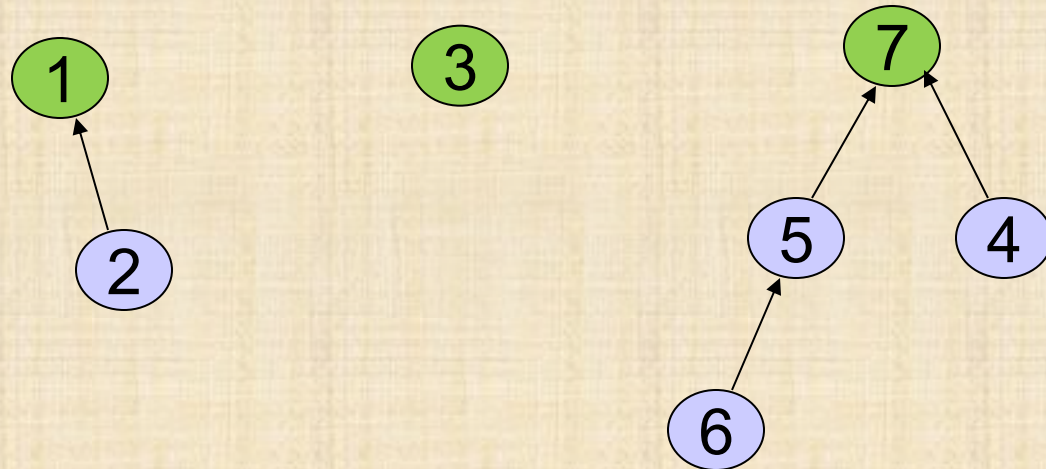


Disjoint-Sets Forest

Initial state



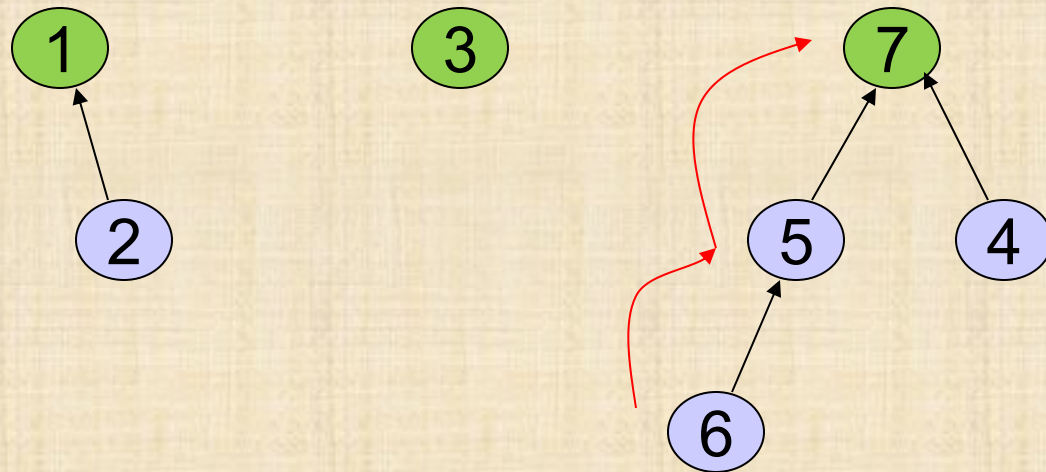
Possible
intermediate
state



Each root is the leader of its set

Find Operation

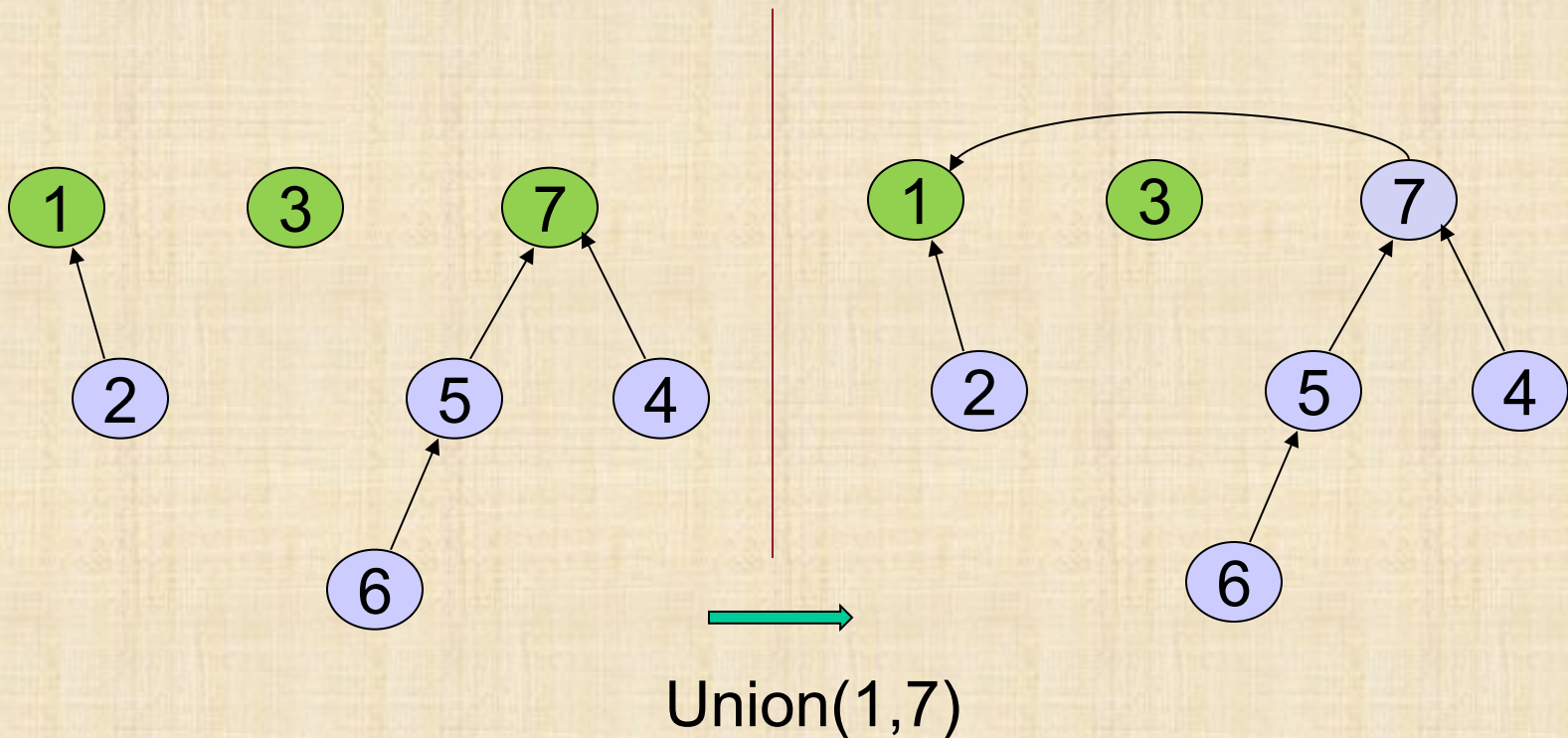
- ▷ *Find(x)* go up from *x* to its tree root and return the root



Find(6) = 7

Union Operation

▷ $Union(i,j)$ [precondition: i and j are roots]:
make j point to i

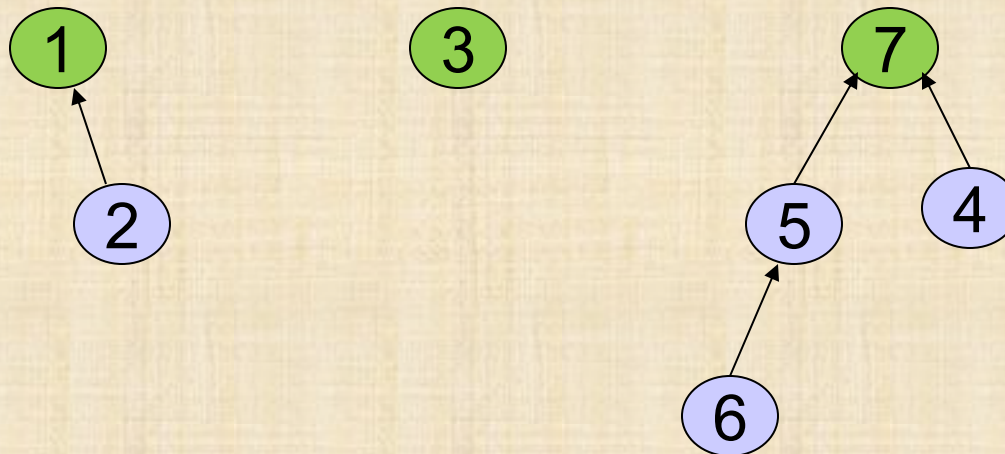


Simple Implementation of *Disjoint-Sets Forest*

- ▷ Array p of parents ($p[i]$ is parent of i)

	1	2	3	4	5	6	7
p	-1	1	-1	7	7	5	-1

For a root x :
 $p[x] = -1$



Union

Union(p,x,y) % precondition: x and y are roots
 $p[y] = x$

constant time

Find

Recursive

```
Find (p, x)
  if  $p[x] = -1$ 
    then return x
  else return Find(p,p[x])
```

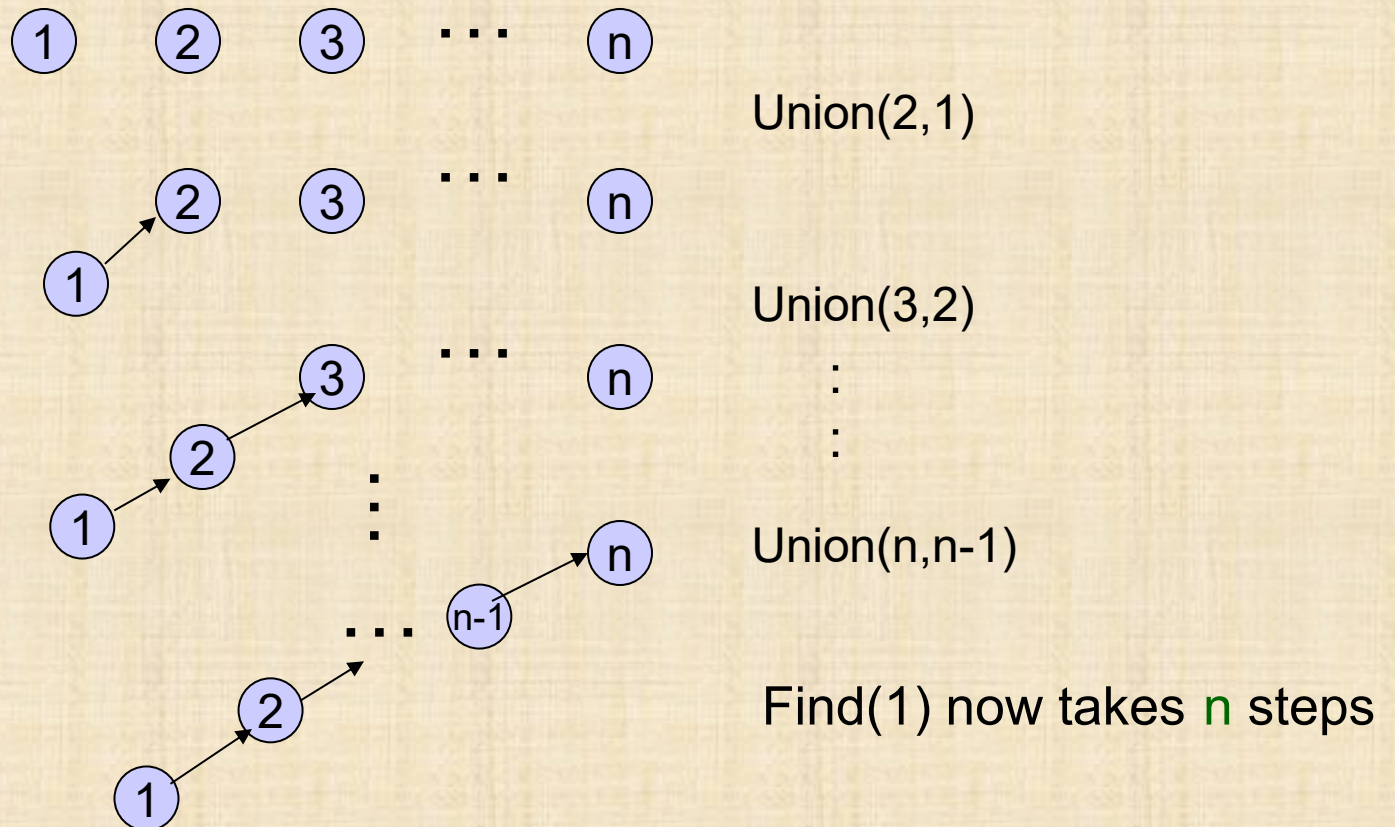
Iterative

```
Find (p, x)
  while  $p[x] \neq -1$  do
     $x = p[x];$ 
  return x;
```

$O(h)$ time

What is the best case?

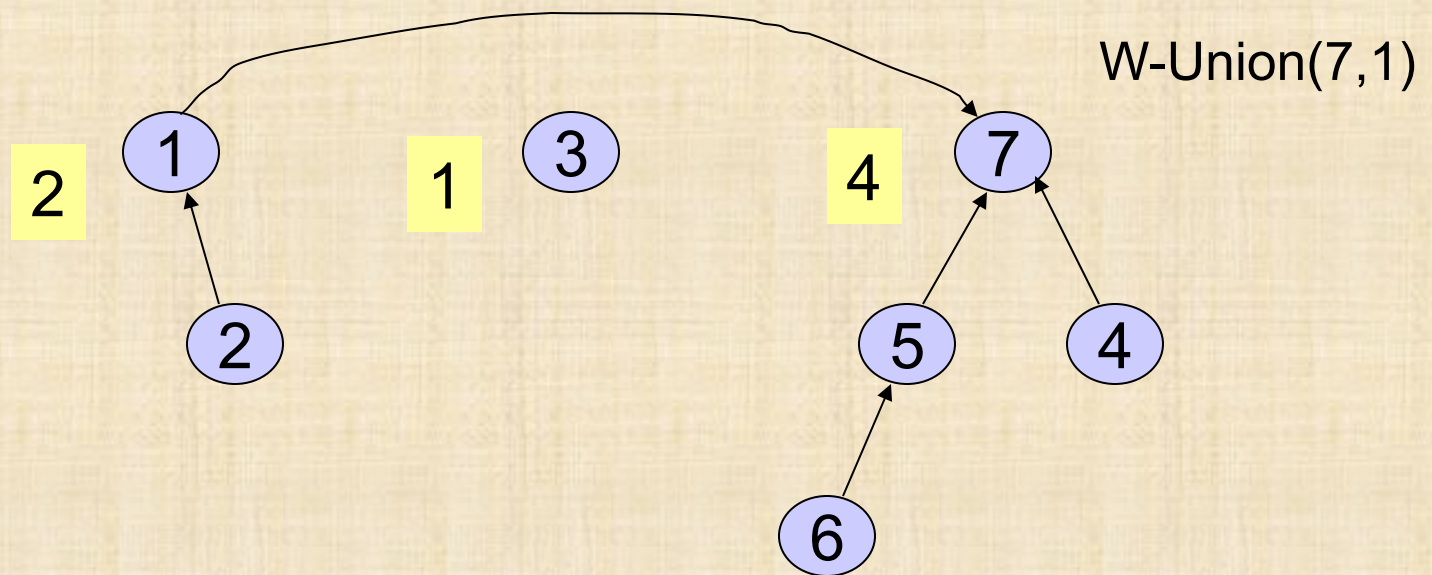
A Bad Case



How can we avoid this bad scenario?
We would like to guarantee $h=O(\log n)$

Union by Weight

- ▷ Weighted Union (**weight** = number of nodes)
 - ▷ Always make the smaller tree point to the root of the larger tree



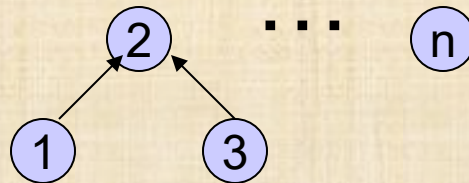
Bad - Example Again



Union(1,2)

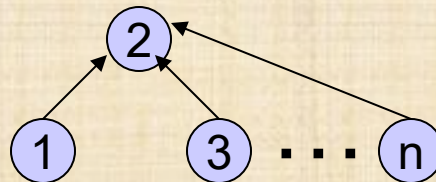


Union(2,3)



⋮

Union($n-1, n$)



Find(1) constant time

Analysis - Depth

Claim: If the depth of x is k then the tree containing x has weight at least 2^k .

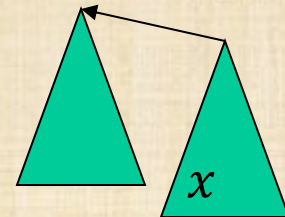
Proof: by induction on k

Base: $k = 0$, x is the only node, so weight is $2^0 = 1$

Inductive assumption: If the depth of x is k then the tree of x has weight at least 2^k .

Inductive step (for $k + 1$):

The depth of x increases from k to $k + 1$ when the root of the tree of x becomes the child of the root of a heavier tree. So after the union, the weight of the tree of x is at least $2^k + 2^k = 2^{k+1}$



Analysis - Depth

Claim: the height of every tree is at most $\log n$

Proof:

Suppose there is some node x whose depth is greater than $\log n$.

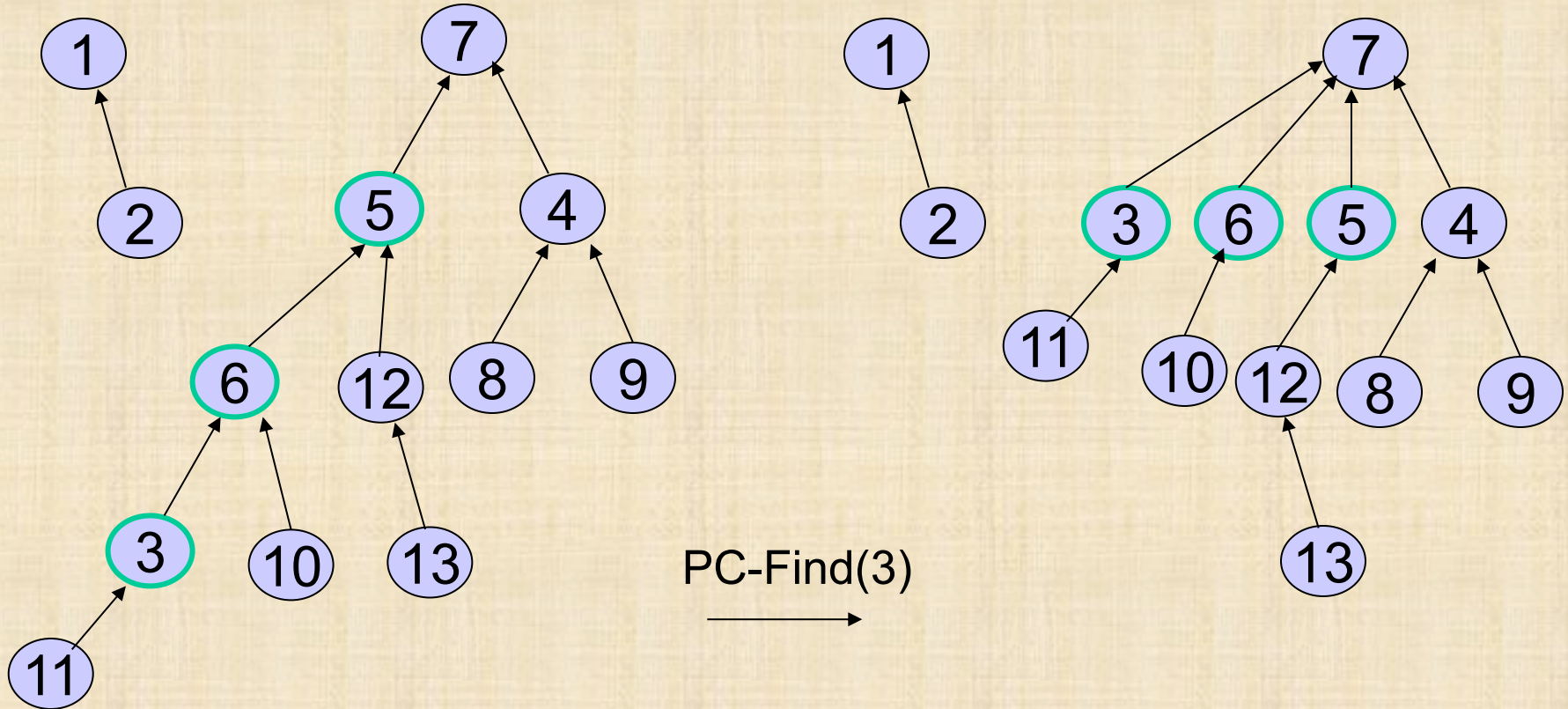
Then the weight of the tree containing x is more than $2^{\log n} = n$, a contradiction.

Time Complexity

- ▷ MakeSet: $O(1)$,
 - ▷ Union: $O(1)$,
 - ▷ Find: $O(\log n)$
-
- ▷ Let's improve further.

Path Compression (PC)

- ▶ On a *Find* operation point all the nodes on the search path directly to the root



Find with Path Compression

PC-Find(i)

```
r = i;  
while p[r]  $\neq$  -1 % i is not root  
    r = p[r];  
if i  $\neq$  r % compress path  
    then k = p[i];  
    while k  $\neq$  r  
        p[i] = r;  
        i = k;  
        k = p[k];  
return(r)
```

iterative

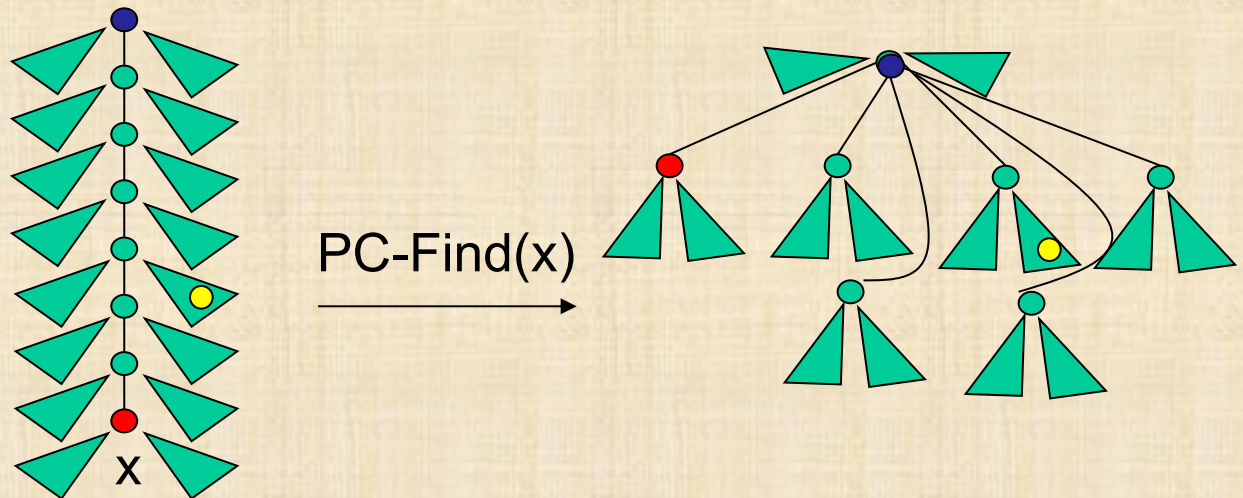
PC-Find(i)

```
r = i  
if p[i]  $\neq$  -1 then % i is not root  
    r = PC-Find(p[i])  
    p[i] = r  
return (r)
```

recursive

Path Compression Works

- ▷ Worst-case time complexity of $Find(x)$ is $O(\log n)$
- ▷ Subsequent *Finds* for some nodes will be very fast
- ▷ Which?



Disjoint-Sets Forest with Union by weight & PC

- ▷ Worst case time complexity:
 - ▷ Union by weight is $O(1)$
 - ▷ Find with path compression is $O(\log n)$
- ▷ Amortized?

$\log^*(n)$

- ▷ $\log^*(n)$ is a function that grows **very** slowly
- ▷ Let's look at the inverse function first: The **tower** function:

$$2 \uparrow\uparrow b = 2^{2^{2^{\cdot^{\cdot^{\cdot}}}}} \quad \begin{matrix} \text{b times} \\ 2 \end{matrix} = \begin{cases} 1 & \text{For } b=0 \\ 2^{2 \uparrow\uparrow (b-1)} & \text{For } b>0 \end{cases}$$

$$2 \uparrow\uparrow 1 = 2$$

$$2 \uparrow\uparrow 2 = 4$$

$$2 \uparrow\uparrow 3 = 16$$

$$2 \uparrow\uparrow 4 = 65536,$$

$$2 \uparrow\uparrow 5 = 2^{65536}$$

$\log^*(n)$

- ▷ $\log^*(n)$ is the inverse of tower
- ▷ $\log^*(n)$ is the number of times we have to take the $\log$ of n until we get ≤ 1

$$\log^* 2 = 1 \quad \text{since } 2 \uparrow \uparrow 1 = 2 ;$$

$$\log^* 4 = 2 \quad \text{since } 2 \uparrow \uparrow 2 = 4 ;$$

$$\log^* 16 = 3 \quad \text{since } 2 \uparrow \uparrow 3 = 16 ;$$

$$\log^* 65536 = 4 \quad \text{since } 2 \uparrow \uparrow 4 = 65536 ;$$

$$\log^* 2^{65536} = 5 \quad \text{since } 2 \uparrow \uparrow 5 = 2^{65536}$$

$$\log \log \log \log \dots \log n \leq 1$$

$\log^*(n)$ times

Union by weight & PC

Theorem (amortized complexity): The worst-case running time of a sequence of m operations is $O(m \log^* n)$

▷ Hence, the amortized complexity is $O(\log^* n)$

Proof: not in this course. See the book.

Summary

- ▷ Four implementations for disjoint U/F:
- ▷ Array - Find: $O(1)$, Union: $O(n)$
- ▷ Linked list - Find: $O(n)$, Union: $O(1)$
- ▷ Up-tree - Find: $O(n)$, Union: $O(1)$
- ▷ Up-tree with union by rank and path compression -
 - ▷ Find: $O(\log n)$ worst case per operation
Union: $O(1)$ worst case per operation
 - ▷ worst case for sequence of m union and find operations $O(m \log^* n)$ (i.e., $O(\log^* n)$ amortized)